

Supplementary Assignment: OOP Hierarchical Design and System Implementation

Object Oriented Programming Language Lab

Course Code: CSE 0613 2134

Introduction

This supplementary assignment provides an opportunity for students to demonstrate mastery of Java OOP concepts covered in **Lab Exam 02 (Object-Oriented Programming)**. The assignment is designed to help students strengthen their understanding of object-oriented programming principles and their practical applications in Java.

By successfully completing this assignment, students can recover marks toward their final Lab Exam grade. The assignment emphasizes practical application of class design, inheritance, and polymorphism in Java and requires demonstrating competence in implementing hierarchical architectures and advanced class organization.

Marking Scheme

Assessment Structure

- **Total Marks Available:** 5 marks
- **Eligibility Requirement:** Students must solve **ALL assigned problems** to receive the full 5 marks
- **Partial Completion:** Incomplete submissions will receive partial credit (determined by the number and weight of correctly implemented problems)

Mark Recovery System

This assignment employs a **recoverable marks** system designed to help students reach the 20-mark ceiling for Lab Exam 02:

Key Principles:

1. The 5 marks from this assignment are added to the Lab Exam 02 score
2. The total cannot exceed **20 marks** (the maximum ceiling)
3. The final score is calculated as: $\min(\text{Lab Exam Score} + \text{Assignment Score}, 20)$

Example Scenarios:

Exam Score	Assignment Score	Final Score
12/20	5/5	17/20
15/20	5/5	20/20
10/20	3/5	13/20
18/20	4/5	20/20
8/20	5/5	13/20

Submission Guidelines

⚠ CRITICAL: Handwritten Submission Required

The following submission requirements are **strict** and **mandatory**:

- ☒ Submission must be **entirely HANDWRITTEN**. No printed code, typed documents, or digital submissions will be accepted.
- ☒ Use **standard A4 paper only** (210mm × 297mm)
- ☒ Write on **ONE SIDE of the paper only** — do NOT use the back of pages
- ☒ Write **ONLY the Problem Number and corresponding Java Code** — do NOT include problem statements, examples, or explanations
- ☒ Handwriting must be **legible and clean**
- ☒ Maintain **strict code indentation and alignment**, even on paper
- ☒ Leave **sufficient white space** between solutions to avoid congestion

Non-compliance with these requirements may result in rejection of the submission or mark deduction.

Content Requirements

Do Include:

- Problem Number (e.g., “Problem 1.1:”, “Problem 2.1:”, etc.)
- Fully functional Java code for each problem
- Proper variable names and comments where appropriate for clarity
- Consistent indentation (use uniform spacing or alignment)

Do NOT Include:

- Problem statement text
- Example input/output
- Algorithm descriptions or explanations
- Any printed or digital elements
- Multiple solutions per page without significant spacing

Format Expectations

Each solution should include:

```
// Problem X.Y: [Title]

// --- Core Implementation ---
// (Define your classes and inheritance hierarchy here)

// --- Driver Class ---
```

Academic Integrity

Code Authenticity

- All submitted code must be **original work** completed by the student
- Copying code from peers, online sources, or AI tools without modification is **academic misconduct**
- Minor similarities in logic are acceptable; significant code duplication will be flagged for review

Plagiarism Policy

- Identical submissions from multiple students will be investigated
- Students found in violation of academic integrity policies will face mark reduction or rejection of assignment

Important Notes

Deadline and Submission Logistics

- Submit your handwritten assignment to the designated class representatives (CR) before the announced deadline
- Late submissions may incur mark deductions or rejection of assignment

Evaluation Criteria

- **Correctness:** Code must execute without errors and produce expected output
- **Completeness:** All assigned problems must be implemented
- **Code Quality:** Proper structure, naming conventions, and readability
- **Handwriting:** Legibility and professional presentation
- **Compliance:** Adherence to all submission guidelines

Summary Checklist

Before submitting, ensure you have completed the following:

- ☐ All assigned problems have been implemented
- ☐ Code is handwritten in clean, legible handwriting
- ☐ A4 paper (one side only) has been used
- ☐ Only Problem Numbers and code are written (no problem statements or examples)
- ☐ Code indentation and alignment are maintained
- ☐ White space is adequate between solutions
- ☐ Work is original and follows academic integrity guidelines
- ☐ Submission deadline has been noted

Good luck with your assignment! This is an excellent opportunity to consolidate your Java programming skills and improve your overall course performance.

Problem 1 : Bank Account Management System

1.1 Problem Statement

A bank needs a software system to manage two types of accounts, Savings Account and Current Account, through a three-level class hierarchy. The abstract root class `BankAccount` defines shared attributes such as account number, holder name, and balance, and declares an abstract method `displayInfo()`.

From `BankAccount`, the abstract class `DepositAccount` adds a deposit-limit concept. `SavingsAccount` and `CurrentAccount` both extend `DepositAccount` as concrete leaf classes.

The class hierarchy is:

```
BankAccount -> DepositAccount -> SavingsAccount / CurrentAccount
```

Savings accounts earn monthly interest, while current accounts support overdraft withdrawals. A static variable `totalAccounts` in `BankAccount` counts every account object created. All fields are private and accessible only through public methods.

To model real-world banking further, `SavingsAccount` contains a nested static class `InterestPolicy` that encapsulates the interest calculation rule. Similarly, `CurrentAccount` contains a nested static class `OverdraftPolicy` that validates whether a withdrawal is within the overdraft limit.

A standalone utility class `InterestCalculator` overloads its `calculate()` method. One version calculates simple interest for one year, and the second version accepts an explicit time parameter in years.

Follow the guidelines provided below to implement all necessary classes, constructors, methods, nested classes, and anything else required to produce the expected output.

1.2 Solving Guidelines

1.2.1 Class Hierarchy

Class Name	Type	Inherits From
<code>BankAccount</code>	abstract class	-
<code>DepositAccount</code>	abstract class	extends <code>BankAccount</code>
<code>SavingsAccount</code>	concrete class	extends <code>DepositAccount</code>
<code>InterestPolicy</code>	static nested class	inside <code>SavingsAccount</code>
<code>CurrentAccount</code>	concrete class	extends <code>DepositAccount</code>
<code>OverdraftPolicy</code>	static nested class	inside <code>CurrentAccount</code>
<code>InterestCalculator</code>	utility class	standalone
<code>BankSystem</code>	Driver Class(main)	-

1.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
BankAccount	accountNumber	String	Instance	private
BankAccount	holderName	String	Instance	private
BankAccount	balance	double	Instance	private
BankAccount	totalAccounts	int	Static	public
DepositAccount	depositLimit	double	Instance	private
SavingsAccount	monthlyInterestRate	double	Instance	private
SavingsAccount	lastInterestApplied	double	Instance	private
CurrentAccount	overdraftLimit	double	Instance	private
CurrentAccount	lastWithdrawalApproved	boolean	Instance	private

1.2.3 Methods

BankAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
getAccountNumber()	String	public	-	Returns private account number
getHolderName()	String	public	-	Returns private holder name
getBalance()	double	public	-	Returns current balance
deposit()	void	public	double amount	Adds positive amount to balance
deductBalance()	void	protected	double amount	Deducts amount from balance
addBalance()	void	protected	double amount	Adds amount to balance
displayInfo()	void	public abstract	-	Forces subclasses to print account details

DepositAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
setDepositLimit()	void	public	double limit	Updates depositLimit
getDepositLimit()	double	public	-	Returns depositLimit

SavingsAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
applyInterest()	void	public	-	Calculates interest using InterestPolicy and adds balance
displayInfo()	void	public	-	Prints savings account details

CurrentAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
withdraw()	void	public	double amount	Checks overdraft policy and deducts amount if allowed
displayInfo()	void	public	-	Prints current account details

Nested Class Methods

Class	Method Name	Return Type	Access	Parameters	Work Done
InterestPolicy	compute()	double	public	double balance, double rate	Returns monthly interest
OverdraftPolicy	isAllowed()	boolean	public	double balance, double amount, double limit	Checks whether withdrawal is allowed

InterestCalculator Methods

Method Name	Return Type	Access	Parameters	Work Done
calculate()	double	public	double principal, double rate	Calculates simple interest for one year
calculate()	double	public	double principal, double rate, int years	Calculates simple interest for given years

1.2.4 Constructors

Class / Nested Class	Constructor Parameters
BankAccount	String accountNumber, String holderName, double balance
DepositAccount	String accountNumber, String holderName, double balance, double depositLimit
SavingsAccount	String accountNumber, String holderName, double balance, double monthlyInterestRate
CurrentAccount	String accountNumber, String holderName, double balance, double overdraftLimit
InterestPolicy	no-arg constructor; static nested class inside SavingsAccount
OverdraftPolicy	no-arg constructor; static nested class inside CurrentAccount
InterestCalculator	no-arg constructor; utility class

1.3 Expected Output

```

=== Savings Account ===
Account No : SA001
Holder    : Alice
Balance   : 52260.0
Monthly Interest Applied: 260.0 (via InterestPolicy)

=== Current Account ===
Account No : CA001

```

Holder	: Bob
Balance	: -500.0
Overdraft	: 1000.0
Withdrawal	DENIED - exceeds overdraft limit (OverdraftPolicy)
Interest (1 yr)	: 3000.0
Interest (3 yr)	: 9000.0
Total Accounts Created	: 2

1.4 Main Function Code

```
01. public class BankSystem {
02.     public static void main(String[] args) {
03.
04.         SavingsAccount sa = new SavingsAccount(
05.             "SA001", "Alice", 50000.0, 0.005
06.         );
07.
08.         CurrentAccount ca = new CurrentAccount(
09.             "CA001", "Bob", 3000.0, 1000.0
10.         );
11.
12.         sa.deposit(2000.0);
13.         sa.applyInterest();
14.         sa.displayInfo();
15.
16.         ca.withdraw(3500.0);
17.         ca.withdraw(5000.0);
18.         ca.displayInfo();
19.
20.         InterestCalculator ic = new InterestCalculator();
21.
22.         System.out.println();
23.         System.out.println("Interest (1 yr) : " + ic.calculate(50000, 6));
24.         System.out.println("Interest (3 yr) : " + ic.calculate(50000, 6, 3));
25.
26.         System.out.println("Total Accounts Created : " + BankAccount.totalAccounts);
27.     }
28. }
```

Problem 2 : Library Book Management System

2.1 Problem Statement

A public library needs a software system to manage two types of books, Physical Books and eBooks, through a structured class hierarchy. The interface `Displayable` declares a method `displayDetails()` that must be implemented by displayable library items.

The abstract root class `LibraryItem` implements `Displayable` and defines common attributes such as item ID, title, and author. From `LibraryItem`, the abstract class `CatalogItem` adds catalog-related information such as catalog number and availability status.

`PhysicalBook` and `EBook` both extend `CatalogItem` as concrete classes.

The class hierarchy is:

```
Displayable -> LibraryItem -> CatalogItem -> PhysicalBook / EBook
```

Physical books contain shelf number and loan period information, while eBooks contain file size and download URL information. A static variable `totalBooks` in `LibraryItem` counts every book object created.

To model real-world library behavior further, `PhysicalBook` contains a nested static class `LoanPolicy` that calculates overdue fines. Similarly, `EBook` contains a nested static class `AccessPolicy` that checks whether a user can download an eBook based on subscription tier.

A standalone utility class `LibrarySearch` overloads its `search()` method. One version searches a book by title, and another version searches a book by item ID.

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, and anything else required to produce the expected output.

2.2 Solving Guidelines

2.2.1 Class Hierarchy

Class Name	Type	Inherits From
<code>Displayable</code>	interface	-
<code>LibraryItem</code>	abstract class	implements <code>Displayable</code>
<code>CatalogItem</code>	abstract class	extends <code>LibraryItem</code>
<code>PhysicalBook</code>	concrete class	extends <code>CatalogItem</code>
<code>LoanPolicy</code>	static nested class	inside <code>PhysicalBook</code>
<code>EBook</code>	concrete class	extends <code>CatalogItem</code>
<code>AccessPolicy</code>	static nested class	inside <code>EBook</code>
<code>LibrarySearch</code>	utility class	standalone
<code>LibrarySystem</code>	Driver Class(main)	-

2.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
LibraryItem	itemID	String	Instance	private
LibraryItem	title	String	Instance	private
LibraryItem	author	String	Instance	private
LibraryItem	totalBooks	int	Static	public
CatalogItem	catalogNumber	String	Instance	private
CatalogItem	isAvailable	boolean	Instance	private
PhysicalBook	shelfNumber	String	Instance	private
PhysicalBook	loanPeriodDays	int	Instance	private
EBook	fileSizeMB	double	Instance	private
EBook	downloadURL	String	Instance	private
LibrarySearch	catalog	LibraryItem[]	Instance	private

2.2.3 Methods

Displayable Method

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public abstract	-	Prints item details

LibraryItem Methods

Method Name	Return Type	Access	Parameters	Work Done
getItemID()	String	public	-	Returns item ID
getTitle()	String	public	-	Returns title
getAuthor()	String	public	-	Returns author
displayDetails()	void	public abstract	-	Prints item details

CatalogItem Methods

Method Name	Return Type	Access	Parameters	Work Done
setAvailable()	void	public	boolean flag	Updates availability
isAvailable()	boolean	public	-	Returns availability
getCatalogNumber()	String	public	-	Returns catalog no

PhysicalBook Methods

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public	-	Prints book details
computeFine()	double	public	int days	Calculates fine

EBook Methods

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public	-	Prints eBook details
canDownload()	boolean	public	String tier	Checks download access

Nested Class Methods

Class	Method Name	Return Type	Access	Parameters	Work Done
LoanPolicy	calculate()	double	public	int days, double rate	Returns fine
AccessPolicy	permitted()	boolean	public	String tier	Checks user access

LibrarySearch Methods

Method Name	Return Type	Access	Parameters	Work Done
search()	void	public	String title	Searches by title
search()	void	public	String itemID, boolean byID	Searches by ID

2.2.4 Constructors

Class / Nested Class	Constructor Parameters
LibraryItem	String itemID, String title, String author
CatalogItem	String itemID, String title, String author, String catalogNumber, boolean isAvailable
PhysicalBook	String itemID, String title, String author, String catalogNumber, boolean isAvailable, String shelfNumber, int loanPeriodDays
EBook	String itemID, String title, String author, String catalogNumber, boolean isAvailable, double fileSizeMB, String downloadURL
LoanPolicy	no-arg constructor; static nested class inside PhysicalBook
AccessPolicy	no-arg constructor; static nested class inside EBook
LibrarySearch	LibraryItem[] catalog

2.3 Expected Output

```

=== Physical Book ===
ID: BK001 | Title: Data Structures | Author: Lipschutz
Shelf: B-05 | Loan Period: 14 days | Available: true
Fine: 50.0

=== EBook ===
ID: EB001 | Title: Clean Code | Author: Martin
Size: 4.5 MB | URL: lib.com/cc | Available: true
PREMIUM: true
BASIC : false

Search by title "Clean Code" -> Found: EB001 - Clean Code
Search by ID "BK001" -> Found: BK001 - Data Structures
Total Books: 2

```

2.4 Main Function Code

```

01. public class LibrarySystem {
02.     public static void main(String[] args) {
03.
04.         PhysicalBook pb = new PhysicalBook(

```

```

05.         "BK001", "Data Structures", "Lipschutz",
06.         "DWY-005", true, "B-05", 14
07.     );
08.
09.     Ebook eb = new Ebook(
10.         "EB001", "Clean Code", "Martin",
11.         "SE-102", true, 4.5, "lib.com/cc"
12.     );
13.
14.     pb.displayDetails();
15.     System.out.println("Fine: " + pb.computeFine(5));
16.
17.     eb.displayDetails();
18.     System.out.println("PREMIUM: " + eb.canDownload("PREMIUM"));
19.     System.out.println("BASIC  : " + eb.canDownload("BASIC"));
20.
21.     LibraryItem[] catalog = {pb, eb};
22.
23.     LibrarySearch ls = new LibrarySearch(catalog);
24.     ls.search("Clean Code");
25.     ls.search("BK001", true);
26.
27.     System.out.println("Total Books: " + LibraryItem.totalBooks);
28. }
29. }

```

Problem 3 : Employee Payroll Processing System

3.1 Problem Statement

TechCorp Ltd. needs a software system to manage employees at different levels. Every employee has an employee ID, name, department, and joining year.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, static nested class, non-static inner class, and static variable tracking.

The abstract class `Employee` stores common employee information and declares abstract methods for salary calculation and employee detail display.

`FullTimeEmployee` extends `Employee` and adds monthly salary and bonus rate.

`SeniorEmployee` extends `FullTimeEmployee` and adds seniority allowance and years of experience.

`PartTimeEmployee` extends `Employee` and adds hourly rate and total hours worked.

The interface `Taxable` declares the method `calculateTax()`.

The class `Employee` contains a static nested class `TaxBracket` that stores tax bracket information.

The class `FullTimeEmployee` contains a non-static inner class `PaySlip` that prints a formatted payslip by accessing the outer employee object.

A static variable `totalEmployees` inside `Employee` tracks the total number of employee objects created.

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, inner classes, and anything else required to produce the expected output.

3.2 Solving Guidelines

3.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Taxable	interface	-
Employee	abstract class	-
TaxBracket	static nested class	inside Employee
FullTimeEmployee	concrete class	extends Employee, implements Taxable
PaySlip	non-static inner class	inside FullTimeEmployee
SeniorEmployee	concrete class	extends FullTimeEmployee
PartTimeEmployee	concrete class	extends Employee, implements Taxable
PayrollSystem	Driver Class(main)	-

3.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Employee	employeeId	String	Instance	private
Employee	name	String	Instance	private
Employee	department	String	Instance	private
Employee	joiningYear	int	Instance	private
Employee	totalEmployees	int	Static	public
FullTimeEmployee	monthlySalary	double	Instance	private
FullTimeEmployee	bonusRate	double	Instance	private
SeniorEmployee	seniorityAllowance	double	Instance	private
SeniorEmployee	yearsOfExperience	int	Instance	private
PartTimeEmployee	hourlyRate	double	Instance	private
PartTimeEmployee	hoursWorked	double	Instance	private
TaxBracket	bracketName	String	Instance	public
TaxBracket	minSalary	double	Instance	public
TaxBracket	taxPercent	double	Instance	public
PaySlip	month	String	Instance	public
PaySlip	remarks	String	Instance	public

3.2.3 Methods

Taxable Method

Method Name	Return Type	Access	Parameters	Work Done
calculateTax()	double	public abstract	-	Calculates tax amount

Employee Methods

Method Name	Return Type	Access	Parameters	Work Done
getEmployeeId()	String	public	-	Returns employee ID
getName()	String	public	-	Returns employee name
getDepartment()	String	public	-	Returns department
getJoiningYear()	int	public	-	Returns joining year
calculateSalary()	double	public abstract	-	Calculates salary
displayDetails()	void	public abstract	-	Prints employee details

TaxBracket Methods

Method Name	Return Type	Access	Parameters	Work Done
describe()	void	public	-	Prints tax bracket info

FullTimeEmployee Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateSalary()	double	public	-	Adds salary and bonus
calculateTax()	double	public	-	Calculates full-time tax
displayDetails()	void	public	-	Prints full-time details

PaySlip Methods

Method Name	Return Type	Access	Parameters	Work Done
printPaySlip()	void	public	-	Prints formatted payslip

SeniorEmployee Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateSalary()	double	public	-	Adds senior allowance
calculateTax()	double	public	-	Calculates senior tax
displayDetails()	void	public	-	Prints senior details

PartTimeEmployee Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateSalary()	double	public	-	Calculates hourly salary
calculateTax()	double	public	-	Calculates part-time tax
displayDetails()	void	public	-	Prints part-time details

3.2.4 Constructors

Class / Nested Class	Constructor Parameters
Employee	String employeeId, String name, String department, int joiningYear
Employee	String employeeId, String name
FullTimeEmployee	employeeId, name, department, joiningYear, monthlySalary, bonusRate
SeniorEmployee	employeeId, name, department, joiningYear, monthlySalary, bonusRate, seniorityAllowance, yearsOfExperience
PartTimeEmployee	employeeId, name, department, joiningYear, hourlyRate, hoursWorked
TaxBracket	String bracketName, double minSalary, double taxPercent
PaySlip	String month, String remarks

3.3 Expected Output

Tax Bracket: High Income | Min Salary: 30000.0 | Tax: 10.0%

=== Senior Employee Details ===

ID : E001

Name : Shirin

Department : Engineering

Joined : 2018

Monthly Salary : 40000.0

Bonus Rate : 10.0%

Seniority Allowance : 8000.0

Years of Experience : 7

```
Net Salary : 52000.0
Tax Amount : 6240.0

--- Pay Slip ---
Employee : Shirin
Month    : June 2025
Salary   : 40000.0 + Bonus: 4000.0 + Seniority: 8000.0
Net Pay  : 52000.0
Remarks : Excellent Performance
```

```
=== Part-Time Employee Details ===
ID       : E002
Name     : Bakar
Department : Sales
Hourly Rate : 200.0
Hours Worked: 60.0
Net Salary : 12000.0
Tax Amount : 600.0
Total Employees: 2
```

3.4 Main Function Code

```
01. public class PayrollSystem {
02.     public static void main(String[] args) {
03.
04.         Employee.TaxBracket tb = new Employee.TaxBracket("High Income", 30000,
05.             10.0);
06.         tb.describe();
07.
08.         SeniorEmployee se = new SeniorEmployee(
09.             "E001", "Shirin", "Engineering", 2018,
10.             40000.0, 10.0, 8000.0, 7
11.         );
12.
13.         PartTimeEmployee pte = new PartTimeEmployee(
14.             "E002", "Bakar", "Sales", 2023,
15.             200.0, 60.0
16.         );
17.         se.displayDetails();
18.         System.out.println("Net Salary : " + se.calculateSalary());
19.         System.out.println("Tax Amount : " + se.calculateTax());
20.
21.         FullTimeEmployee.PaySlip ps = se.new PaySlip("June 2025", "Excellent
22.             Performance");
23.         ps.printPaySlip();
24.         pte.displayDetails();
25.         System.out.println("Net Salary : " + pte.calculateSalary());
```

```
26.         System.out.println("Tax Amount : " + pte.calculateTax());
27.
28.         System.out.println("Total Employees: " + Employee.totalEmployees);
29.     }
30. }
```

Problem 4 : E-Commerce Product Order System

4.1 Problem Statement

An online store needs a software system to manage two types of products, Electronics and Clothing, through a structured class hierarchy. The interface `Orderable` declares a method `placeOrder()` that must be implemented by orderable products.

The abstract root class `Item` stores common item information such as item ID, name, and price. It also declares an abstract method `computeFinalPrice()`.

From `Item`, the abstract class `Product` is derived. It adds product category and stock quantity information.

`Electronics` and `Clothing` both extend `Product` as concrete classes.

The class hierarchy is:

```
Orderable -> Item -> Product -> Electronics / Clothing
```

`Electronics` products have brand and warranty period information. `Clothing` products have size and material information.

A static variable `totalProducts` in `Item` counts every product object created.

To model real-world pricing rules, `Electronics` contains a nested static class `WarrantyPricing` that calculates warranty surcharge. Similarly, `Clothing` contains a nested static class `SeasonalDiscount` that applies seasonal discount.

A standalone utility class `DiscountEngine` overloads its `applyDiscount()` method. One version applies a flat discount, and another version applies a percentage discount.

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, and anything else required to produce the expected output.

4.2 Solving Guidelines

4.2.1 Class Hierarchy

Class Name	Type	Inherits From
Orderable	interface	-
Item	abstract class	-
Product	abstract class	extends Item, implements Orderable
Electronics	concrete class	extends Product
WarrantyPricing	static nested class	inside Electronics
Clothing	concrete class	extends Product
SeasonalDiscount	static nested class	inside Clothing
DiscountEngine	utility class	standalone
ShopNow	Driver Class(main)	-

4.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Item	itemID	String	Instance	private
Item	name	String	Instance	private
Item	price	double	Instance	private
Item	totalProducts	int	Static	public
Product	category	String	Instance	private
Product	stockQuantity	int	Instance	private
Electronics	brand	String	Instance	private
Electronics	warrantyMonths	int	Instance	private
Clothing	size	String	Instance	private
Clothing	material	String	Instance	private

4.2.3 Methods

Orderable Method

Method Name	Return Type	Access	Parameters	Work Done
placeOrder()	void	public abstract	-	Places product order

Item Methods

Method Name	Return Type	Access	Parameters	Work Done
getItemID()	String	public	-	Returns item ID
getName()	String	public	-	Returns item name
getPrice()	double	public	-	Returns item price
computeFinalPrice()	double	public abstract	-	Calculates final price
displayDetails()	void	public	-	Prints basic details

Product Methods

Method Name	Return Type	Access	Parameters	Work Done
getCategory()	String	public	-	Returns category
getStockQuantity()	int	public	-	Returns stock quantity
reduceStock()	void	public	int qty	Reduces stock quantity
placeOrder()	void	public abstract	-	Places product order

Electronics Methods

Method Name	Return Type	Access	Parameters	Work Done
computeFinalPrice()	double	public	-	Adds warranty surcharge
placeOrder()	void	public	-	Places electronics order
displayDetails()	void	public	-	Prints electronics info

Clothing Methods

Method Name	Return Type	Access	Parameters	Work Done
computeFinalPrice()	double	public	-	Applies discount
placeOrder()	void	public	-	Places clothing order
displayDetails()	void	public	-	Prints clothing info

Nested Class Methods

Class	Method Name	Return Type	Access	Parameters	Work Done
WarrantyPricing	surcharge()	double	public	int warrantyMonths	Returns surcharge
SeasonalDiscount	apply()	double	public	double price, double rate	Returns final price

DiscountEngine Methods

Method Name	Return Type	Access	Parameters	Work Done
applyDiscount()	double	public	double price, double flatAmount	Applies flat discount
applyDiscount()	double	public	double price, double percent, boolean isPercent	Applies percent discount

4.2.4 Constructors

Class / Nested Class	Constructor Parameters
Item	String itemID, String name, double price
Product	String itemID, String name, double price, String category, int stockQuantity
Electronics	itemID, name, price, category, stockQuantity, brand, warrantyMonths
Clothing	itemID, name, price, category, stockQuantity, size, material
WarrantyPricing	no-arg constructor; static nested class inside Electronics
SeasonalDiscount	no-arg constructor; static nested class inside Clothing
DiscountEngine	no-arg constructor; utility class

4.3 Expected Output

```

=== Electronics ===
ID: E001 | Name: Laptop | Category: Electronics | Base: 55000.0
Brand: Dell | Warranty: 24 months
WarrantyPricing surcharge: 1200.0
Final Price: 56200.0
Order Placed! Stock: 10 -> 9

=== Clothing ===
ID: C001 | Name: T-Shirt | Category: Clothing | Base: 500.0
Size: M | Material: Cotton
SeasonalDiscount (10%): 450.0
Final Price: 450.0

```

Order Placed! Stock: 50 -> 49

Flat : 53000.0

Percent : 450.0

Total Products: 2

4.4 Main Function Code

```
01. public class ShopNow {
02.     public static void main(String[] args) {
03.
04.         Electronics elec = new Electronics(
05.             "E001", "Laptop", 55000, "Electronics", 10, "Dell", 24
06.         );
07.
08.         Clothing cloth = new Clothing(
09.             "C001", "T-Shirt", 500, "Clothing", 50, "M", "Cotton"
10.         );
11.
12.         elec.displayDetails();
13.         System.out.println("Final Price: " + elec.computeFinalPrice());
14.         elec.placeOrder();
15.
16.         cloth.displayDetails();
17.         System.out.println("Final Price: " + cloth.computeFinalPrice());
18.         cloth.placeOrder();
19.
20.         DiscountEngine de = new DiscountEngine();
21.
22.         System.out.println("Flat : " + de.applyDiscount(55000, 2000));
23.         System.out.println("Percent : " + de.applyDiscount(500, 10, true));
24.
25.         System.out.println("Total Products: " + Item.totalProducts);
26.     }
27. }
```

Problem 5 : University Course Registration System

5.1 Problem Statement

SUST needs a software system to manage different types of university courses. Every course has a course code, course title, credit hours, instructor name, seat limit, and enrolled student count.

The system should be designed using abstraction, inheritance, interface implementation, method overriding, constructor chaining, static nested class, non-static inner class, and static variable tracking.

The interface `Schedulable` declares a method `generateSchedule()` that must be implemented by schedulable courses.

The abstract class `Course` implements `Schedulable` and stores common course information. It also declares abstract methods for calculating weekly contact hours and displaying course details.

`TheoryCourse` extends `Course` and adds lecture hours per week.

`AdvancedTheoryCourse` extends `TheoryCourse` and adds prerequisite course code and research component information.

`LabCourse` extends `Course` and adds lab hours per week and required equipment.

The class `Course` contains a static nested class `TimeSlot` that stores day, start time, and end time information.

The class `TheoryCourse` contains a non-static inner class `Enrollment` that stores student ID and enrollment date. It directly accesses the outer course object to print enrollment confirmation.

A static variable `totalCourses` inside `Course` tracks the total number of course objects created.

The class hierarchy is:

Course -> TheoryCourse -> AdvancedTheoryCourse
Course -> LabCourse
Course implements Schedulable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, inner classes, and anything else required to produce the expected output.

5.2 Solving Guidelines

5.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
<code>Schedulable</code>	interface	-
<code>Course</code>	abstract class	implements <code>Schedulable</code>
<code>TimeSlot</code>	static nested class	inside <code>Course</code>
<code>TheoryCourse</code>	concrete class	extends <code>Course</code>
<code>Enrollment</code>	non-static inner class	inside <code>TheoryCourse</code>
<code>AdvancedTheoryCourse</code>	concrete class	extends <code>TheoryCourse</code>
<code>LabCourse</code>	concrete class	extends <code>Course</code>
<code>UniversitySystem</code>	Driver Class(main)	-

5.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Course	courseCode	String	Instance	private
Course	courseTitle	String	Instance	private
Course	creditHours	int	Instance	private
Course	instructorName	String	Instance	private
Course	seatLimit	int	Instance	private
Course	enrolledCount	int	Instance	private
Course	totalCourses	int	Static	public
TheoryCourse	lectureHrsPerWeek	int	Instance	private
AdvancedTheoryCourse	prerequisiteCourse	String	Instance	private
AdvancedTheoryCourse	hasResearchComponent	boolean	Instance	private
LabCourse	labHrsPerWeek	int	Instance	private
LabCourse	equipment	String	Instance	private
TimeSlot	day	String	Instance	public
TimeSlot	startTime	String	Instance	public
TimeSlot	endTime	String	Instance	public
Enrollment	studentId	String	Instance	public
Enrollment	enrollmentDate	String	Instance	public

5.2.3 Methods

Schedulable Method

Method Name	Return Type	Access	Parameters	Work Done
generateSchedule()	void	public abstract	-	Prints course schedule

Course Methods

Method Name	Return Type	Access	Parameters	Work Done
getCourseCode()	String	public	-	Returns course code
getCourseTitle()	String	public	-	Returns course title
getAvailableSeats()	int	public	-	Returns available seats
registerStudent()	void	public	-	Registers one student
contactHours()	int	public abstract	-	Calculates contact hrs
displayDetails()	void	public abstract	-	Prints course details

TimeSlot Methods

Method Name	Return Type	Access	Parameters	Work Done
display()	void	public	-	Prints time slot

TheoryCourse Methods

Method Name	Return Type	Access	Parameters	Work Done
contactHours()	int	public	-	Returns lecture hours
generateSchedule()	void	public	-	Prints theory schedule
displayDetails()	void	public	-	Prints theory details

Enrollment Methods

Method Name	Return Type	Access	Parameters	Work Done
printConfirmation()	void	public	-	Prints enrollment confirmation

AdvancedTheoryCourse Methods

Method Name	Return Type	Access	Parameters	Work Done
contactHours()	int	public	-	Adds research hours
generateSchedule()	void	public	-	Prints advanced schedule
displayDetails()	void	public	-	Prints advanced details

LabCourse Methods

Method Name	Return Type	Access	Parameters	Work Done
contactHours()	int	public	-	Returns lab hours
generateSchedule()	void	public	-	Prints lab schedule
displayDetails()	void	public	-	Prints lab details

5.2.4 Constructors

Class / Nested Class	Constructor Parameters
Course	courseCode, courseTitle, creditHours, instructorName, seatLimit
TheoryCourse	courseCode, courseTitle, creditHours, instructorName, seatLimit, lectureHrsPerWeek
AdvancedTheoryCourse	courseCode, courseTitle, creditHours, instructorName, seatLimit, lectureHrsPerWeek, prerequisiteCourse, hasResearchComponent
LabCourse	courseCode, courseTitle, creditHours, instructorName, seatLimit, labHrsPerWeek, equipment
TimeSlot	String day, String startTime, String endTime
Enrollment	String studentId, String enrollmentDate

5.3 Expected Output

Time Slot: Monday | 09:00 - 12:00

=== Advanced Theory Course Details ===

Course Code : CSE101

Title : Advanced 00P

Credit Hours: 3

Instructor : Dr. Rahman

Prerequisite: CSE101

Research : Yes

Lecture/week: 3 hours

```
Student registered. (1/30)
Student registered. (2/30)
Schedule: CSE101 | Mon-Wed-Fri | 3+2 hrs/day | Lecture Hall + Research Lab
Contact hrs/week : 5
Available Seats : 28
```

```
--- Enrollment Confirmation ---
```

```
Student : STU-001
Course  : CSE101 - Advanced OOP
Date    : 2025-06-01
```

```
=== Lab Course Details ===
```

```
Course Code : CSE101L
Title       : OOP Lab
Credit Hours: 1
Instructor  : Dr. Rahman
Lab hrs/week: 2
Equipment   : PC, JDK, IDE
Schedule: CSE101L | Tue-Thu | 2 hrs/day | Lab (PC, JDK, IDE)
Total Courses: 2
```

5.4 Main Function Code

```
01. public class UniversitySystem {
02.     public static void main(String[] args) {
03.
04.         Course.TimeSlot ts = new Course.TimeSlot("Monday", "09:00", "12:00");
05.         ts.display();
06.
07.         AdvancedTheoryCourse atc = new AdvancedTheoryCourse(
08.             "CSE101", "Advanced OOP", 3, "Dr. Rahman",
09.             30, 3, "CSE101", true
10.         );
11.
12.         LabCourse lc = new LabCourse(
13.             "CSE101L", "OOP Lab", 1, "Dr. Rahman",
14.             20, 2, "PC, JDK, IDE"
15.         );
16.
17.         atc.displayDetails();
18.         atc.registerStudent();
19.         atc.registerStudent();
20.         atc.generateSchedule();
21.
22.         System.out.println("Contact hrs/week : " + atc.contactHours());
23.         System.out.println("Available Seats : " + atc.getAvailableSeats());
24.
25.         TheoryCourse.Enrollment enr = atc.new Enrollment("STU-001", "2025-06-01");
```

```
26.         enr.printConfirmation();
27.
28.         lc.displayDetails();
29.         lc.generateSchedule();
30.
31.         System.out.println("Total Courses: " + Course.totalCourses);
32.     }
33. }
```

Problem 6 : Transport Vehicle Fare Calculation System

6.1 Problem Statement

MoveEasy Transport Co. needs a software system to manage different types of vehicles, such as Bus, Luxury Bus, and Taxi.

Every vehicle has a vehicle ID, route name, and base fare.

The system should be designed using abstraction, inheritance, method overriding, method overloading, static nested class, non-static inner class, and static variable tracking.

The abstract class `Vehicle` stores common vehicle information and declares abstract methods for fare calculation and vehicle information display.

`Bus` extends `Vehicle` and adds seat capacity and per-kilometer charge.

`LuxuryBus` extends `Bus` and adds comfort surcharge and lounge access information.

`Taxi` extends `Vehicle` and adds per-kilometer rate and waiting charge per minute.

The class `Vehicle` contains a static nested class `Route` that stores route origin and destination.

The class `Bus` contains a non-static inner class `FareSummary` that stores the final fare and passenger count for a trip.

A static variable `totalVehicles` inside `Vehicle` tracks the total number of vehicle objects created.

The class hierarchy is:

```
Vehicle -> Bus -> LuxuryBus
```

```
Vehicle -> Taxi
```

The system must support:

- displaying route information
- displaying vehicle information
- calculating fares using method overriding
- calculating group-discounted bus fare using method overloading
- calculating taxi fare with and without waiting time
- printing fare summary using inner class
- tracking total vehicles using a static variable

Follow the guidelines provided below to implement all necessary classes, constructors, methods, nested classes, inner classes, and anything else required to produce the expected output.

6.2 Solving Guidelines

6.2.1 Class Hierarchy

Class Name	Type	Inherits From
Vehicle	abstract class	-
Route	static nested class	inside Vehicle
Bus	concrete class	extends Vehicle
FareSummary	non-static inner class	inside Bus
LuxuryBus	concrete class	extends Bus
Taxi	concrete class	extends Vehicle
TransportSystem	Driver Class(main)	-

6.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Vehicle	vehicleId	String	Instance	private
Vehicle	routeName	String	Instance	private
Vehicle	baseFare	double	Instance	private
Vehicle	totalVehicles	int	Static	public
Bus	seatCapacity	int	Instance	private
Bus	chargePerKm	double	Instance	private
LuxuryBus	comfortSurcharge	double	Instance	private
LuxuryBus	hasLounge	boolean	Instance	private
Taxi	ratePerKm	double	Instance	private
Taxi	waitingChargePerMin	double	Instance	private
Route	origin	String	Instance	public
Route	destination	String	Instance	public
FareSummary	totalFare	double	Instance	public
FareSummary	passengerCount	int	Instance	public

6.2.3 Methods

Vehicle Methods

Method Name	Return Type	Access	Parameters	Work Done
getVehicleId()	String	public	-	Returns vehicle ID
getRouteName()	String	public	-	Returns route name
getBaseFare()	double	public	-	Returns base fare
calculateFare()	double	public abstract	double km	Calculates fare
displayInfo()	void	public abstract	-	Prints vehicle info

Route Methods

Method Name	Return Type	Access	Parameters	Work Done
display()	void	public	-	Prints route info

Bus Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateFare()	double	public	double km	Calculates bus fare
calculateFare()	double	public	double km, int pax	Applies group discount
displayInfo()	void	public	-	Prints bus info

FareSummary Methods

Method Name	Return Type	Access	Parameters	Work Done
printSummary()	void	public	-	Prints fare summary

LuxuryBus Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateFare()	double	public	double km	Adds comfort surcharge
calculateFare()	double	public	double km, int pax	Applies group discount
displayInfo()	void	public	-	Prints luxury bus info

Taxi Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateFare()	double	public	double km	Calculates taxi fare
calculateFare()	double	public	double km, int waitMin	Adds waiting charge
displayInfo()	void	public	-	Prints taxi info

6.2.4 Constructors

Class / Nested Class	Constructor Parameters
Vehicle	String vehicleId, String routeName, double baseFare
Bus	vehicleId, routeName, baseFare, seatCapacity, chargePerKm
LuxuryBus	vehicleId, routeName, baseFare, seatCapacity, chargePerKm, comfortSurcharge, hasLounge
Taxi	vehicleId, routeName, baseFare, ratePerKm, waitingChargePerMin
Route	String origin, String destination
FareSummary	double totalFare, int passengerCount

6.3 Expected Output

Route: Sylhet -> Dhaka
=== Bus Info ===
Vehicle ID : BUS01
Route : Sylhet-Dhaka
Base Fare : 50.0
Seat Capacity : 45
Charge/km : 5.0
Bus fare (20km) : 150.0
Bus fare (20km,35 pax): 142.5
--- Fare Summary ---
Total Fare : 142.5
Passenger Count: 35
=== Luxury Bus Info ===
Vehicle ID : LB01
Comfort Surcharge: 200.0
Lounge Access : Yes
Luxury fare (20km) : 400.0
=== Taxi Info ===
Vehicle ID : TAX01

Rate/km	: 10.0
Waiting/min	: 2.0
Taxi fare (15km)	: 180.0
Taxi fare (15km,10min wait):	200.0
Total Vehicles:	3

6.4 Main Function Code

```

01. public class TransportSystem {
02.     public static void main(String[] args) {
03.
04.         Vehicle.Route r1 = new Vehicle.Route("Sylhet", "Dhaka");
05.
06.         Bus b = new Bus("BUS01", "Sylhet-Dhaka", 50.0, 45, 5.0);
07.
08.         LuxuryBus lb = new LuxuryBus(
09.             "LB01", "Sylhet-Dhaka", 80.0,
10.             30, 6.0, 200.0, true
11.         );
12.
13.         Taxi t = new Taxi("TAX01", "City Route", 30.0, 10.0, 2.0);
14.
15.         r1.display();
16.
17.         b.displayInfo();
18.         System.out.println("Bus fare (20km)          : " + b.calculateFare(20));
19.         System.out.println("Bus fare (20km,35 pax): " + b.calculateFare(20, 35));
20.
21.         Bus.FareSummary fs = b.new FareSummary(b.calculateFare(20, 35), 35);
22.         fs.printSummary();
23.
24.         lb.displayInfo();
25.         System.out.println("Luxury fare (20km)      : " + lb.calculateFare(20));
26.
27.         t.displayInfo();
28.         System.out.println("Taxi fare (15km)          : " + t.calculateFare(15));
29.         System.out.println("Taxi fare (15km,10min wait): " + t.calculateFare(15,
30.             10));
31.         System.out.println("Total Vehicles: " + Vehicle.totalVehicles);
32.     }
33. }

```

Problem 7 : Hospital Patient Management System

7.1 Problem Statement

A hospital needs a software system to manage two types of patients, InPatients and OutPatients, through a three-level class hierarchy.

The abstract root class `Patient` stores common patient information such as patient ID, name, and age. It also declares an abstract method `computeBill()`.

From `Patient`, the abstract class `RegisteredPatient` is derived. It adds registration date and assigned doctor information.

`InPatient` and `OutPatient` both extend `RegisteredPatient` as concrete classes.

The class hierarchy is:

```
Patient -> RegisteredPatient -> InPatient / OutPatient
```

InPatients have ward number, daily charge, and number of days admitted. OutPatients have consultation fee and prescription cost.

A static variable `totalPatients` in `Patient` counts every patient object created.

To model real-world billing rules, `InPatient` contains a nested static class `WardBillingPolicy` that calculates ward cost. Similarly, `OutPatient` contains a nested static class `PrescriptionChargePolicy` that calculates prescription charge with a fixed dispensing fee.

A standalone utility class `MedicalTaxCalculator` overloads its `computeTax()` method. One version calculates tax using income only, and another version calculates tax after adjusting income based on number of dependents.

Follow the guidelines provided below to implement all necessary classes, constructors, methods, nested classes, and anything else required to produce the expected output.

7.2 Solving Guidelines

7.2.1 Class Hierarchy

Class Name	Type	Inherits From
Patient	abstract class	-
RegisteredPatient	abstract class	extends Patient
InPatient	concrete class	extends RegisteredPatient
WardBillingPolicy	static nested class	inside InPatient
OutPatient	concrete class	extends RegisteredPatient
PrescriptionChargePolicy	static nested class	inside OutPatient
MedicalTaxCalculator	utility class	standalone
HospitalSystem	Driver Class(main)	-

7.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Patient	patientID	String	Instance	private
Patient	name	String	Instance	private
Patient	age	int	Instance	private
Patient	totalPatients	int	Static	public
RegisteredPatient	registrationDate	String	Instance	private
RegisteredPatient	assignedDoctor	String	Instance	private
InPatient	wardNumber	int	Instance	private
InPatient	dailyCharge	double	Instance	private
InPatient	daysAdmitted	int	Instance	private
OutPatient	consultationFee	double	Instance	private
OutPatient	prescriptionCost	double	Instance	private

7.2.3 Methods

Patient Methods

Method Name	Return Type	Access	Parameters	Work Done
displayInfo()	void	public	-	Prints patient info
computeBill()	double	public abstract	-	Calculates bill

RegisteredPatient Methods

Method Name	Return Type	Access	Parameters	Work Done
getAssignedDoctor()	String	public	-	Returns assigned doctor
getRegistrationDate()	String	public	-	Returns registration date

InPatient Methods

Method Name	Return Type	Access	Parameters	Work Done
computeBill()	double	public	-	Calculates ward bill
displayInfo()	void	public	-	Prints inpatient info

OutPatient Methods

Method Name	Return Type	Access	Parameters	Work Done
computeBill()	double	public	-	Calculates OPD bill
displayInfo()	void	public	-	Prints outpatient info

Nested Class Methods

Class	Method Name	Return Type	Access	Parameters	Work Done
WardBillingPolicy	compute()	double	public	double dailyCharge, int days	Returns ward cost
PrescriptionChargePolicy	total()	double	public	double prescriptionCost	Returns medicine cost

MedicalTaxCalculator Methods

Method Name	Return Type	Access	Parameters	Work Done
computeTax()	double	public	double income	Calculates normal tax
computeTax()	double	public	double income, int deps	Calculates adjusted tax

7.2.4 Constructors

Class / Nested Class	Constructor Parameters
Patient	String patientID, String name, int age
RegisteredPatient	patientID, name, age, registrationDate, assignedDoctor
InPatient	patientID, name, age, registrationDate, assignedDoctor, wardNumber, dailyCharge, daysAdmitted
OutPatient	patientID, name, age, registrationDate, assignedDoctor, consultationFee, prescriptionCost
WardBillingPolicy	no-arg constructor; static nested class inside InPatient
PrescriptionChargePolicy	no-arg constructor; static nested class inside OutPatient
MedicalTaxCalculator	no-arg constructor; utility class

7.3 Expected Output

```
=== InPatient ===
ID: P001 | Name: Rafi | Age: 45
Doctor: Dr. Alam | Registered: 01-06-2024
Ward: 3 | Daily: 1500.0 | Days: 7
Ward Cost (WardBillingPolicy): 10500.0
Bill: 10500.0

=== OutPatient ===
ID: P002 | Name: Mim | Age: 30
Doctor: Dr. Roy | Registered: 02-06-2024
Consultation: 300.0
Prescription + Dispensing (PrescriptionChargePolicy): 470.0
Bill: 770.0

Tax (no dep) : 1575.0
Tax (2 dep) : 1425.0
Total Patients: 2
```

7.4 Main Function Code

```
01. public class HospitalSystem {
02.     public static void main(String[] args) {
03.
04.         InPatient ip = new InPatient(
05.             "P001", "Rafi", 45, "01-06-2024", "Dr. Alam",
06.             3, 1500.0, 7
07.         );
08.     }
```

```
09.         OutPatient op = new OutPatient(  
10.             "P002", "Mim", 30, "02-06-2024", "Dr. Roy",  
11.             300.0, 450.0  
12.         );  
13.  
14.         ip.displayInfo();  
15.         System.out.println("Bill: " + ip.computeBill());  
16.  
17.         op.displayInfo();  
18.         System.out.println("Bill: " + op.computeBill());  
19.  
20.         MedicalTaxCalculator mtc = new MedicalTaxCalculator();  
21.  
22.         System.out.println("Tax (no dep) : " + mtc.computeTax(10500));  
23.         System.out.println("Tax (2 dep) : " + mtc.computeTax(10500, 2));  
24.  
25.         System.out.println("Total Patients: " + Patient.totalPatients);  
26.     }  
27. }
```

Problem 8 : Restaurant Food Order Management System

8.1 Problem Statement

TastyCo Restaurant needs a software system to manage different types of menu items, such as Chef Special food items and Beverage items.

Every menu item has an item code, item name, and unit price.

The system should be designed using abstraction, inheritance, method overriding, method overloading, static nested class, non-static inner class, and static variable tracking.

The abstract class MenuItem stores common menu item information and declares abstract methods for displaying item details and calculating order total.

FoodItem extends MenuItem and adds cuisine type and preparation time.

ChefSpecial extends FoodItem and adds chef name and signature discount percentage.

BeverageItem extends MenuItem and adds temperature and volume information.

The class MenuItem contains a static nested class Category that stores category code and category description.

The class FoodItem contains a non-static inner class OrderTicket that stores table number and quantity. It directly accesses the outer food item object to print an order ticket.

A static variable totalItems inside MenuItem tracks the total number of menu item objects created.

The class hierarchy is:

```
MenuItem -> FoodItem -> ChefSpecial
```

```
MenuItem -> BeverageItem
```

The system must support:

- displaying category information
- displaying menu item details
- calculating order total
- applying discount using method overloading
- applying chef special discount using method overriding
- printing order ticket using inner class
- tracking total menu items using a static variable

Follow the guidelines provided below to implement all necessary classes, constructors, methods, nested classes, inner classes, and anything else required to produce the expected output.

8.2 Solving Guidelines

8.2.1 Class Hierarchy

Class Name	Type	Inherits From
MenuItem	abstract class	-
Category	static nested class	inside MenuItem
FoodItem	concrete class	extends MenuItem
OrderTicket	non-static inner class	inside FoodItem
ChefSpecial	concrete class	extends FoodItem
BeverageItem	concrete class	extends MenuItem
RestaurantSystem	Driver Class(main)	-

8.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
MenuItem	itemCode	String	Instance	private
MenuItem	itemName	String	Instance	private
MenuItem	unitPrice	double	Instance	private
MenuItem	totalItems	int	Static	public
FoodItem	cuisineType	String	Instance	private
FoodItem	prepTimeMins	int	Instance	private
ChefSpecial	chefName	String	Instance	private
ChefSpecial	signatureDiscount	double	Instance	private
BeverageItem	temperature	String	Instance	private
BeverageItem	volumeMl	int	Instance	private
Category	categoryCode	String	Instance	public
Category	description	String	Instance	public
OrderTicket	tableNumber	int	Instance	public
OrderTicket	quantity	int	Instance	public

8.2.3 Methods

MenuItem Methods

Method Name	Return Type	Access	Parameters	Work Done
getItemCode()	String	public	-	Returns item code
getItemName()	String	public	-	Returns item name
getUnitPrice()	double	public	-	Returns unit price
orderTotal()	double	public	int quantity	Calculates total
orderTotal()	double	public	int quantity, double disc	Applies discount
displayDetails()	void	public abstract	-	Prints item details

Category Methods

Method Name	Return Type	Access	Parameters	Work Done
display()	void	public	-	Prints category info

FoodItem Methods

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public	-	Prints food details

OrderTicket Methods

Method Name	Return Type	Access	Parameters	Work Done
printTicket()	void	public	-	Prints order ticket

ChefSpecial Methods

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public	-	Prints special details
orderTotal()	double	public	int quantity	Applies special disc

BeverageItem Methods

Method Name	Return Type	Access	Parameters	Work Done
displayDetails()	void	public	-	Prints beverage info

8.2.4 Constructors

Class / Nested Class	Constructor Parameters
MenuItem	String itemCode, String itemName, double unitPrice
FoodItem	itemCode, itemName, unitPrice, cuisineType, prepTimeMins
ChefSpecial	itemCode, itemName, unitPrice, cuisineType, prepTimeMins, chefName, signatureDiscount
BeverageItem	itemCode, itemName, unitPrice, temperature, volumeMl
Category	String categoryCode, String description
OrderTicket	int tableNumber, int quantity

8.3 Expected Output

Category: MAIN | Main Course

=== Chef Special Details ===

Item Code : CS01

Name : Hilsa Curry

Unit Price : 400.0

Cuisine : Bengali

Prep Time : 25 mins

Chef : Chef Rahim

Signature Disc: 10.0%

ChefSpecial total (3 qty) : 1080.0

Regular total (3 qty, 5% off): 1140.0

--- Order Ticket ---

Table : 7

Item : CS01 - Hilsa Curry

Price : 400.0 x 3 = 1080.0

=== Beverage Details ===

Item Code : BV01

Name : Mango Lassi

Unit Price: 120.0

Temperature: Cold

Volume : 300 ml

Beverage total (5 qty) : 600.0

Total Items: 2

8.4 Main Function Code

```
01. public class RestaurantSystem {
02.     public static void main(String[] args) {
03.
04.         MenuItem.Category cat = new MenuItem.Category("MAIN", "Main Course");
05.         cat.display();
06.
07.         ChefSpecial cs = new ChefSpecial(
08.             "CS01", "Hilsa Curry", 400.0, "Bengali",
09.             25, "Chef Rahim", 10.0
10.         );
11.
12.         BeverageItem b = new BeverageItem(
13.             "BV01", "Mango Lassi", 120.0, "Cold", 300
14.         );
15.
16.         cs.displayDetails();
17.
18.         System.out.println("ChefSpecial total (3 qty) : " +
19.             cs.orderTotal(3));
20.         System.out.println("Regular total (3 qty, 5% off): " + cs.orderTotal(3,
21.             5.0));
22.
23.         FoodItem.OrderTicket ot = cs.new OrderTicket(7, 3);
24.         ot.printTicket();
25.
26.         b.displayDetails();
27.         System.out.println("Beverage total (5 qty) : " + b.orderTotal(5));
28.
29.         System.out.println("Total Items: " + MenuItem.totalItems);
30.     }
31. }
```

Problem 9 : School Student Grade Management System

9.1 Problem Statement

A school needs a software system to manage two types of students, Undergraduate Students and Graduate Students, through a structured class hierarchy.

The interface `Evaluable` declares a method `computeGrade()` that must be implemented by evaluable student classes.

The abstract root class `Person` stores common personal information such as person ID, name, and email.

From `Person`, the abstract class `Student` is derived. It adds student ID, program, and marks. `Student` also implements the `Evaluable` interface.

`UndergraduateStudent` and `GraduateStudent` both extend `Student` as concrete classes.

The class hierarchy is:

```
Person -> Student -> UndergraduateStudent / GraduateStudent
```

```
Student implements Evaluable
```

Undergraduate students have year of study information. Graduate students have thesis title and supervisor information.

A static variable `totalStudents` inside `Person` tracks the total number of student objects created.

To model grading rules, `UndergraduateStudent` contains a nested static class `UGGradePolicy` that assigns grades for undergraduate students. Similarly, `GraduateStudent` contains a nested static class `GradGradePolicy` that assigns grades for graduate students.

A standalone utility class `GradeCalculator` overloads its `evaluate()` method. One version evaluates grade from average marks, and another version evaluates grade after adding a bonus mark.

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, and anything else required to produce the expected output.

9.2 Solving Guidelines

9.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
<code>Evaluable</code>	interface	-
<code>Person</code>	abstract class	-
<code>Student</code>	abstract class	extends <code>Person</code> , implements <code>Evaluable</code>
<code>UndergraduateStudent</code>	concrete class	extends <code>Student</code>
<code>UGGradePolicy</code>	static nested class	inside <code>UndergraduateStudent</code>
<code>GraduateStudent</code>	concrete class	extends <code>Student</code>
<code>GradGradePolicy</code>	static nested class	inside <code>GraduateStudent</code>
<code>GradeCalculator</code>	utility class	standalone
<code>SchoolSystem</code>	Driver Class(main)	-

9.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Person	personID	String	Instance	private
Person	name	String	Instance	private
Person	email	String	Instance	private
Person	totalStudents	int	Static	public
Student	studentID	String	Instance	private
Student	program	String	Instance	private
Student	marks	double[]	Instance	private
UndergraduateStudent	yearOfStudy	int	Instance	private
GraduateStudent	thesisTitle	String	Instance	private
GraduateStudent	supervisor	String	Instance	private

9.2.3 Methods

Evaluable Method

Method Name	Return Type	Access	Parameters	Work Done
computeGrade()	String	public abstract	-	Computes grade

Person Methods

Method Name	Return Type	Access	Parameters	Work Done
getPersonID()	String	public	-	Returns person ID
getName()	String	public	-	Returns name
getEmail()	String	public	-	Returns email
displayInfo()	void	public	-	Prints person info

Student Methods

Method Name	Return Type	Access	Parameters	Work Done
getStudentID()	String	public	-	Returns student ID
getProgram()	String	public	-	Returns program
getAverage()	double	public	-	Calculates average
computeGrade()	String	public abstract	-	Computes grade
displayInfo()	void	public	-	Prints student info

UndergraduateStudent Methods

Method Name	Return Type	Access	Parameters	Work Done
computeGrade()	String	public	-	Applies UG policy
displayInfo()	void	public	-	Prints UG details

GraduateStudent Methods

Method Name	Return Type	Access	Parameters	Work Done
computeGrade()	String	public	-	Applies grad policy
displayInfo()	void	public	-	Prints graduate details

Nested Class Methods

Class	Method Name	Return Type	Access	Parameters	Work Done
UGGradePolicy	assign()	String	public	double average	Returns UG grade
GradGradePolicy	assign()	String	public	double average	Returns grad grade

GradeCalculator Methods

Method Name	Return Type	Access	Parameters	Work Done
evaluate()	String	public	double average	Evaluates grade
evaluate()	String	public	double average, double bonus	Evaluates bonus grade

9.2.4 Constructors

Class / Nested Class	Constructor Parameters
Person	String personID, String name, String email
Student	personID, name, email, studentID, program, marks
UndergraduateStudent	personID, name, email, studentID, program, marks, yearOfStudy
GraduateStudent	personID, name, email, studentID, program, marks, thesisTitle, supervisor
UGGradePolicy	no-arg constructor; static nested class inside UndergraduateStudent
GradGradePolicy	no-arg constructor; static nested class inside GraduateStudent
GradeCalculator	no-arg constructor; utility class

9.3 Expected Output

```
=== Undergraduate Student ===
ID: UG01 | Name: Karim | Email: karim@uni.edu
Roll: S001 | Program: CSE | Year: 3
Average: 81.67 | Grade (UGGradePolicy): A
Grade: A

=== Graduate Student ===
ID: GR01 | Name: Nila | Email: nila@uni.edu
Roll: S002 | Program: MS-CSE
Thesis: AI Ethics | Supervisor: Dr. Smith
Average: 88.33 | Grade (GradGradePolicy): A
Grade: A

evaluate(81.67)      : A
evaluate(81.67,5.0) : A+
Total Students: 2
```

9.4 Main Function Code

```
01. public class SchoolSystem {
02.     public static void main(String[] args) {
03.
04.         double[] m1 = {75, 80, 90};
```

```

05.         double[] m2 = {85, 88, 92};
06.
07.         UndergraduateStudent ug = new UndergraduateStudent(
08.             "UG01", "Karim", "karim@uni.edu",
09.             "S001", "CSE", m1, 3
10.         );
11.
12.         GraduateStudent grad = new GraduateStudent(
13.             "GR01", "Nila", "nila@uni.edu",
14.             "S002", "MS-CSE", m2,
15.             "AI Ethics", "Dr. Smith"
16.         );
17.
18.         ug.displayInfo();
19.         System.out.println("Grade: " + ug.computeGrade());
20.
21.         grad.displayInfo();
22.         System.out.println("Grade: " + grad.computeGrade());
23.
24.         GradeCalculator gc = new GradeCalculator();
25.
26.         System.out.println("evaluate(81.67)      : " + gc.evaluate(81.67));
27.         System.out.println("evaluate(81.67,5.0) : " + gc.evaluate(81.67, 5.0));
28.
29.         System.out.println("Total Students: " + Person.totalStudents);
30.     }
31. }

```

Problem 10 : Smart Home Appliance Control System

10.1 Problem Statement

SmartLiving Co. needs a software system to manage different types of smart home appliances, such as Smart Lighting Devices and Climate Devices.

Every appliance has a device ID, brand, power consumption in watts, and ON/OFF status.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, static nested class, non-static inner class, and static variable tracking.

The interface `Controllable` declares two methods, `turnOn()` and `turnOff()`, that must be implemented by controllable appliances.

The abstract class `Appliance` implements `Controllable` and stores common appliance information.

`LightingDevice` extends `Appliance` and adds brightness level and color information.

`SmartLightingDevice` extends `LightingDevice` and adds voice control and automation schedule information.

`ClimateDevice` extends `Appliance` and adds temperature and mode information.

The class `Appliance` contains a static nested class `DeviceSpec` that stores model number and manufacturing year.

The class `LightingDevice` contains a non-static inner class `SceneConfig` that stores scene name and target brightness. It directly accesses the outer lighting device object to apply a lighting scene.

A static variable `totalDevices` inside `Appliance` tracks the total number of appliance objects created.

The class hierarchy is:

```
Appliance -> LightingDevice -> SmartLightingDevice
```

```
Appliance -> ClimateDevice
```

```
Appliance implements Controllable
```

The system must support:

- displaying device specification
- turning appliances on and off
- displaying appliance status
- calculating energy usage
- applying lighting scenes using inner class
- tracking total devices using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, nested classes, inner classes, and anything else required to produce the expected output.

10.2 Solving Guidelines

10.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Controllable	interface	-
Appliance	abstract class	implements Controllable
DeviceSpec	static nested class	inside Appliance
LightingDevice	concrete class	extends Appliance
SceneConfig	non-static inner class	inside LightingDevice
SmartLightingDevice	concrete class	extends LightingDevice
ClimateDevice	concrete class	extends Appliance
SmartHomeSystem	Driver Class(main)	-

10.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Appliance	deviceId	String	Instance	private
Appliance	brand	String	Instance	private
Appliance	powerWatts	double	Instance	private
Appliance	isOn	boolean	Instance	private
Appliance	totalDevices	int	Static	public
LightingDevice	brightnessLevel	int	Instance	private
LightingDevice	color	String	Instance	private
SmartLightingDevice	voiceControlEnabled	boolean	Instance	private
SmartLightingDevice	automationSchedule	String	Instance	private
ClimateDevice	temperatureC	double	Instance	private
ClimateDevice	mode	String	Instance	private
DeviceSpec	modelName	String	Instance	public
DeviceSpec	manufacturingYear	int	Instance	public
SceneConfig	sceneName	String	Instance	public
SceneConfig	targetBrightness	int	Instance	public

10.2.3 Methods

Controllable Methods

Method Name	Return Type	Access	Parameters	Work Done
turnOn()	void	public abstract	-	Turns device on
turnOff()	void	public abstract	-	Turns device off

Appliance Methods

Method Name	Return Type	Access	Parameters	Work Done
getDeviceId()	String	public	-	Returns device ID
getBrand()	String	public	-	Returns brand
getPowerWatts()	double	public	-	Returns power watts
isActive()	boolean	public	-	Returns ON/OFF state
energyUsed()	double	public abstract	double hours	Calculates energy
displayStatus()	void	public abstract	-	Prints device status

DeviceSpec Methods

Method Name	Return Type	Access	Parameters	Work Done
display()	void	public	-	Prints device spec

LightingDevice Methods

Method Name	Return Type	Access	Parameters	Work Done
turnOn()	void	public	-	Turns light on
turnOff()	void	public	-	Turns light off
energyUsed()	double	public	double hours	Calculates light energy
displayStatus()	void	public	-	Prints light status

SceneConfig Methods

Method Name	Return Type	Access	Parameters	Work Done
applyScene()	void	public	-	Applies scene

SmartLightingDevice Methods

Method Name	Return Type	Access	Parameters	Work Done
turnOn()	void	public	-	Turns smart light on
turnOff()	void	public	-	Turns smart light off
energyUsed()	double	public	double hours	Applies energy saving
displayStatus()	void	public	-	Prints smart light info

ClimateDevice Methods

Method Name	Return Type	Access	Parameters	Work Done
turnOn()	void	public	-	Turns climate device on
turnOff()	void	public	-	Turns climate device off
energyUsed()	double	public	double hours	Calculates climate energy
displayStatus()	void	public	-	Prints climate status

10.2.4 Constructors

Class / Nested Class	Constructor Parameters
Appliance	String deviceId, String brand, double powerWatts
Appliance	String deviceId, String brand
LightingDevice	deviceId, brand, powerWatts, brightnessLevel, color
SmartLightingDevice	deviceId, brand, powerWatts, brightnessLevel, color, voiceControlEnabled, automationSchedule
ClimateDevice	deviceId, brand, powerWatts, temperatureC, mode
DeviceSpec	String modelNumber, int manufacturingYear
SceneConfig	String sceneName, int targetBrightness

10.3 Expected Output

```
Device Spec | Model: PH-L900 | Year: 2024
SmartLightingDevice L001 (Philips) is ON. [Voice Control: Enabled]

=== Smart Lighting Device Status ===
Device ID      : L001
Brand          : Philips
Power          : 10.0 W
Status         : ON
Brightness     : 75%
Color          : Warm White
Voice Control  : Enabled
Automation Schedule : 22:00 OFF / 06:00 ON
Energy (8 hrs): 0.068 kWh

--- Scene Config ---
Scene Name     : Movie Mode
Target Bright  : 30%
Current Color  : Warm White
Current Bright : 75% -> applying 30%

SmartLightingDevice L001 (Philips) is OFF.
ClimateDevice C001 (Gree) is ON.

=== Climate Device Status ===
Device ID      : C001
Brand          : Gree
Power          : 1500.0 W
Status         : ON
Temperature    : 24.0 C
Mode           : Cool
Energy (5 hrs): 9.0 kWh
ClimateDevice C001 (Gree) is OFF.
Total Devices: 2
```

10.4 Main Function Code

```
01. public class SmartHomeSystem {
02.     public static void main(String[] args) {
03.
04.         Appliance.DeviceSpec spec = new Appliance.DeviceSpec("PH-L900", 2024);
05.         spec.display();
06.
07.         SmartLightingDevice sld = new SmartLightingDevice(
08.             "L001", "Philips", 10.0, 75,
09.             "Warm White", true, "22:00 OFF / 06:00 ON"
10.         );
11.
12.         ClimateDevice cd = new ClimateDevice(
13.             "C001", "Gree", 1500.0, 24.0, "Cool"
14.         );
15.
16.         sld.turnOn();
17.         sld.displayStatus();
18.
19.         System.out.println("Energy (8 hrs): " + sld.energyUsed(8) + " kWh");
20.
21.         LightingDevice.SceneConfig scene = sld.new SceneConfig("Movie Mode", 30);
22.         scene.applyScene();
23.
24.         sld.turnOff();
25.
26.         cd.turnOn();
27.         cd.displayStatus();
28.
29.         System.out.println("Energy (5 hrs): " + cd.energyUsed(5) + " kWh");
30.
31.         cd.turnOff();
32.
33.         System.out.println("Total Devices: " + Appliance.totalDevices);
34.     }
35. }
```

Problem 11 : Airline Reservation System

11.1 Problem Statement

SkyWings Airlines needs a software system to manage different types of passengers, such as Premium Passengers and Corporate Passengers.

Every person has a person ID, full name, and nationality.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Boardable` declares a method `generateBoardingPass()` that must be implemented by boardable passengers.

The abstract class `Person` stores common personal information.

The abstract class `Passenger` extends `Person` and implements `Boardable`. It adds passport number and booked seat count.

`PremiumPassenger` extends `Passenger` and adds membership tier and loyalty miles.

`CorporatePassenger` extends `PremiumPassenger` and adds company name and corporate discount.

A standalone class `Flight` stores flight information such as flight code, destination, base fare per seat, and total seats.

A static variable `totalBookings` inside `Passenger` tracks total confirmed bookings. In the given main function, no `bookSeats()` method is called, so total bookings remain zero.

The class hierarchy is:

```
Person -> Passenger -> PremiumPassenger -> CorporatePassenger
```

```
Passenger implements Boardable
```

The system must support:

- displaying flight information
- displaying passenger information
- calculating ticket fare
- calculating multi-seat fare using method overloading
- applying premium and corporate discounts
- generating boarding passes
- tracking total bookings using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

11.2 Solving Guidelines

11.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Boardable	interface	-
Person	abstract class	-
Passenger	abstract class	extends Person, implements Boardable
PremiumPassenger	concrete class	extends Passenger
CorporatePassenger	concrete class	extends PremiumPassenger
Flight	standalone class	-
AirlineSystem	Driver Class(main)	-

11.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Person	personId	String	Instance	private
Person	fullName	String	Instance	private
Person	nationality	String	Instance	private
Passenger	passportNumber	String	Instance	private
Passenger	seatsBooked	int	Instance	private
Passenger	totalBookings	int	Static	public
PremiumPassenger	membershipTier	String	Instance	private
PremiumPassenger	loyaltyMiles	double	Instance	private
CorporatePassenger	companyName	String	Instance	private
CorporatePassenger	corporateDiscount	double	Instance	private
Flight	flightCode	String	Instance	private
Flight	destination	String	Instance	private
Flight	baseFarePerSeat	double	Instance	private
Flight	totalSeats	int	Instance	private

11.2.3 Methods

Boardable Method

Method Name	Return Type	Access	Parameters	Work Done
generateBoardingPass()	void	public abstract	-	Prints boarding pass

Person Methods

Method Name	Return Type	Access	Parameters	Work Done
getPersonId()	String	public	-	Returns person ID
getFullName()	String	public	-	Returns full name
getNationality()	String	public	-	Returns nationality
displayInfo()	void	public abstract	-	Prints person info

Passenger Methods

Method Name	Return Type	Access	Parameters	Work Done
getPassportNumber()	String	public	-	Returns passport no
getSeatsBooked()	int	public	-	Returns seat count
bookSeats()	void	public	int n	Books seats
calculateFare()	double	public abstract	Flight f	Calculates one fare
calculateFare()	double	public abstract	Flight f, int seats	Calculates multi fare
displayInfo()	void	public abstract	-	Prints passenger info

PremiumPassenger Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateFare()	double	public	Flight f	Applies premium discount
calculateFare()	double	public	Flight f, int seats	Calculates multi fare
addMiles()	void	public	double miles	Adds loyalty miles
generateBoardingPass()	void	public	-	Prints premium pass
displayInfo()	void	public	-	Prints premium info

CorporatePassenger Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateFare()	double	public	Flight f	Applies corporate discount
calculateFare()	double	public	Flight f, int seats	Calculates multi fare
generateBoardingPass()	void	public	-	Prints corporate pass
displayInfo()	void	public	-	Prints corporate info

Flight Methods

Method Name	Return Type	Access	Parameters	Work Done
getFlightCode()	String	public	-	Returns flight code
getDestination()	String	public	-	Returns destination
getBaseFarePerSeat()	double	public	-	Returns base fare
displayFlightInfo()	void	public	-	Prints flight info

11.2.4 Constructors

Class	Constructor Parameters
Person	String personId, String fullName, String nationality
Person	String personId, String fullName
Passenger	personId, fullName, nationality, passportNumber
PremiumPassenger	personId, fullName, nationality, passportNumber, membershipTier
CorporatePassenger	personId, fullName, nationality, passportNumber, membershipTier, companyName, corporateDiscount
Flight	String flightCode, String destination, double baseFarePerSeat, int totalSeats

11.3 Expected Output

```
=== Flight Info ===
Flight: SW201 | Destination: Dubai
Fare/Seat: 15000.0 | Seats: 180

=== Corporate Passenger ===
ID: P001 | Name: Arif | Nationality: Bangladeshi
Passport: BD123456 | Tier: Gold
Company: SUST Corp | Corp Discount: 15.0%
Corp Fare(1): 11475.0
Corp Fare(3): 34425.0

*** CORPORATE BOARDING PASS ***
Passenger: Arif | Flight: SW201 | Dest: Dubai | Company: SUST Corp

=== Premium Passenger ===
ID: P002 | Name: Sadia | Tier: Silver
Loyalty Miles: 0.0
Prem Fare(2): 27000.0
Loyalty Miles after add: 500.0

*** PREMIUM BOARDING PASS ***
Passenger: Sadia | Flight: N/A | Miles: 500.0
Total Bookings: 0
```

11.4 Main Function Code

```
01. public class AirlineSystem {
02.     public static void main(String[] args) {
03.
04.         Flight f = new Flight("SW201", "Dubai", 15000.0, 180);
05.
06.         CorporatePassenger cp = new CorporatePassenger(
07.             "P001", "Arif", "Bangladeshi",
08.             "BD123456", "Gold", "SUST Corp", 15.0
09.         );
10.
11.         PremiumPassenger pp = new PremiumPassenger(
12.             "P002", "Sadia", "Bangladeshi",
13.             "BD654321", "Silver"
14.         );
15.
16.         f.displayFlightInfo();
17.
18.         cp.displayInfo();
19.         System.out.println("Corp Fare(1): " + cp.calculateFare(f));
20.         System.out.println("Corp Fare(3): " + cp.calculateFare(f, 3));
21.         cp.generateBoardingPass();
```

```
22.  
23.     pp.displayInfo();  
24.     System.out.println("Prem Fare(2): " + pp.calculateFare(f, 2));  
25.     pp.addMiles(500.0);  
26.     pp.generateBoardingPass();  
27.  
28.     System.out.println("Total Bookings: " + Passenger.totalBookings);  
29. }  
30. }
```

Problem 12 : Insurance Policy Management System

12.1 Problem Statement

SecureLife Insurance needs a software system to manage different types of policyholders, such as Life Insured individuals and Senior Life Insured individuals.

Every entity has an entity ID, full name, and contact number.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Claimable` declares a method `processClaim()` that must be implemented by claimable policyholders.

The abstract class `Entity` stores common identity information.

The abstract class `PolicyHolder` extends `Entity` and implements `Claimable`. It adds age and policy number.

`LifeInsured` extends `PolicyHolder` and adds nominee name and nominee relation.

`SeniorLifeInsured` extends `LifeInsured` and adds medical allowance and special benefit status.

A standalone class `InsurancePolicy` stores policy information such as policy type, coverage amount, and base premium rate.

A static variable `totalPolicies` inside `PolicyHolder` tracks the total number of policyholder objects created.

The class hierarchy is:

```
Entity -> PolicyHolder -> LifeInsured -> SeniorLifeInsured
```

```
PolicyHolder implements Claimable
```

The system must support:

- displaying insurance policy information
- displaying policyholder profiles
- calculating annual premium
- calculating multi-year premium using method overloading
- applying senior medical allowance
- processing insurance claims
- tracking total policies using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

12.2 Solving Guidelines

12.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Claimable	interface	-
Entity	abstract class	-
PolicyHolder	abstract class	extends Entity, implements Claimable
LifeInsured	concrete class	extends PolicyHolder
SeniorLifeInsured	concrete class	extends LifeInsured
InsurancePolicy	standalone class	-
InsuranceSystem	Driver Class(main)	-

12.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Entity	entityId	String	Instance	private
Entity	fullName	String	Instance	private
Entity	contactNumber	String	Instance	private
PolicyHolder	age	int	Instance	private
PolicyHolder	policyNumber	String	Instance	private
PolicyHolder	totalPolicies	int	Static	public
LifeInsured	nomineeName	String	Instance	private
LifeInsured	nomineeRelation	String	Instance	private
SeniorLifeInsured	medicalAllowance	double	Instance	private
SeniorLifeInsured	hasSpecialBenefit	boolean	Instance	private
InsurancePolicy	policyType	String	Instance	private
InsurancePolicy	coverageAmount	double	Instance	private
InsurancePolicy	basePremiumRate	double	Instance	private

12.2.3 Methods

Claimable Method

Method Name	Return Type	Access	Parameters	Work Done
processClaim()	void	public abstract	-	Processes claim

Entity Methods

Method Name	Return Type	Access	Parameters	Work Done
getEntityId()	String	public	-	Returns entity ID
getFullName()	String	public	-	Returns full name
getContactNumber()	String	public	-	Returns contact no
displayProfile()	void	public abstract	-	Prints profile

PolicyHolder Methods

Method Name	Return Type	Access	Parameters	Work Done
getAge()	int	public	-	Returns age
getPolicyNumber()	String	public	-	Returns policy number
calculatePremium()	double	public abstract	InsurancePolicy p	Calculates premium
calculatePremium()	double	public abstract	InsurancePolicy p, int yrs	Calculates multi-year premium
displayProfile()	void	public abstract	-	Prints profile

LifeInsured Methods

Method Name	Return Type	Access	Parameters	Work Done
calculatePremium()	double	public	InsurancePolicy p	Calculates life premium
calculatePremium()	double	public	InsurancePolicy p, int yrs	Calculates multi-year premium
processClaim()	void	public	-	Processes life claim
displayProfile()	void	public	-	Prints life profile

SeniorLifeInsured Methods

Method Name	Return Type	Access	Parameters	Work Done
calculatePremium()	double	public	InsurancePolicy p	Adds medical allowance
calculatePremium()	double	public	InsurancePolicy p, int yrs	Calculates senior multi-year premium
processClaim()	void	public	-	Processes senior claim
displayProfile()	void	public	-	Prints senior profile

InsurancePolicy Methods

Method Name	Return Type	Access	Parameters	Work Done
getPolicyType()	String	public	-	Returns policy type
getCoverageAmount()	double	public	-	Returns coverage
getBasePremiumRate()	double	public	-	Returns rate
displayPolicy()	void	public	-	Prints policy info

12.2.4 Constructors

Class	Constructor Parameters
Entity	String entityId, String fullName, String contactNumber
Entity	String entityId, String fullName
PolicyHolder	entityId, fullName, contactNumber, age, policyNumber
LifeInsured	entityId, fullName, contactNumber, age, policyNumber, nomineeName, nomineeRelation
SeniorLifeInsured	entityId, fullName, contactNumber, age, policyNumber, nomineeName, nomineeRelation, medicalAllowance
InsurancePolicy	String policyType, double coverageAmount, double basePremiumRate

12.3 Expected Output

```
=== Insurance Policy ===
Type      : Life
Coverage  : 500000.0
Rate      : 2.0%

=== Life Insured Profile ===
ID: E001 | Name: Nasrin | Contact: 01711000001
Age: 45 | Policy: POL001
Nominee: Rahim (Spouse)

Annual Prem : 10500.0
5-yr Prem   : 52500.0
Processing Life Insured Claim for Nasrin...
Claim forwarded to Nominee: Rahim

=== Senior Life Insured Profile ===
ID: E002 | Name: Kamal | Age: 68 | Policy: POL002
Nominee: Sara (Child)
Medical Allowance: 3000.0 | Special Benefit: YES

Senior Prem : 13500.0
Processing SENIOR Claim for Kamal...
Special Benefit activated! Medical Allowance: 3000.0
Total Policies: 2
```

12.4 Main Function Code

```
01. public class InsuranceSystem {
02.     public static void main(String[] args) {
03.
04.         InsurancePolicy ip = new InsurancePolicy("Life", 500000.0, 0.02);
05.
06.         LifeInsured li = new LifeInsured(
07.             "E001", "Nasrin", "01711000001",
08.             45, "POL001", "Rahim", "Spouse"
```

```

09.         );
10.
11.         SeniorLifeInsured si = new SeniorLifeInsured(
12.             "E002", "Kamal", "01822000002",
13.             68, "POL002", "Sara", "Child", 3000.0
14.         );
15.
16.         ip.displayPolicy();
17.
18.         li.displayProfile();
19.         System.out.println("Annual Prem : " + li.calculatePremium(ip));
20.         System.out.println("5-yr Prem   : " + li.calculatePremium(ip, 5));
21.         li.processClaim();
22.
23.         si.displayProfile();
24.         System.out.println("Senior Prem : " + si.calculatePremium(ip));
25.         si.processClaim();
26.
27.         System.out.println("Total Policies: " + PolicyHolder.totalPolicies);
28.     }
29. }

```

Problem 13 : Real Estate Property Listing System

13.1 Problem Statement

HomeFinder Realty needs a software system to manage different types of properties, such as Residential Properties and Luxury Residences.

Every asset has an asset ID, owner name, and registration date.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Listable` declares a method `generateListing()` that must be implemented by listable properties.

The abstract class `Asset` stores common asset information.

The abstract class `Property` extends `Asset` and implements `Listable`. It adds location, area in square feet, and base price.

`ResidentialProperty` extends `Property` and adds number of bedrooms, number of bathrooms, and parking availability.

`LuxuryResidence` extends `ResidentialProperty` and adds pool availability, gym availability, and luxury markup percentage.

A standalone class `PropertyAgent` stores agent information and calculates commission.

A static variable `totalListings` inside `Property` tracks the total number of property objects created.

The class hierarchy is:

```
Asset -> Property -> ResidentialProperty -> LuxuryResidence
```

```
Property implements Listable
```

The system must support:

- displaying property information
- generating property listing cards
- calculating sale price
- calculating discounted price using method overloading
- calculating luxury residence price using markup
- calculating agent commission
- tracking total listed properties using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

13.2 Solving Guidelines

13.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Listable	interface	-
Asset	abstract class	-
Property	abstract class	extends Asset, implements Listable
ResidentialProperty	concrete class	extends Property
LuxuryResidence	concrete class	extends ResidentialProperty
PropertyAgent	standalone class	-
RealEstateSystem	Driver Class(main)	-

13.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Asset	assetId	String	Instance	private
Asset	ownerName	String	Instance	private
Asset	registrationDate	String	Instance	private
Property	location	String	Instance	private
Property	areaSqFt	double	Instance	private
Property	basePrice	double	Instance	private
Property	totalListings	int	Static	public
ResidentialProperty	bedrooms	int	Instance	private
ResidentialProperty	bathrooms	int	Instance	private
ResidentialProperty	hasParking	boolean	Instance	private
LuxuryResidence	hasPool	boolean	Instance	private
LuxuryResidence	hasGym	boolean	Instance	private
LuxuryResidence	luxuryMarkupPercent	double	Instance	private
PropertyAgent	agentId	String	Instance	private
PropertyAgent	agentName	String	Instance	private
PropertyAgent	commissionRate	double	Instance	private

13.2.3 Methods

Listable Method

Method Name	Return Type	Access	Parameters	Work Done
generateListing()	void	public abstract	-	Prints listing card

Asset Methods

Method Name	Return Type	Access	Parameters	Work Done
getAssetId()	String	public	-	Returns asset ID
getOwnerName()	String	public	-	Returns owner name
displayAssetInfo()	void	public abstract	-	Prints asset info

Property Methods

Method Name	Return Type	Access	Parameters	Work Done
getLocation()	String	public	-	Returns location
getAreaSqFt()	double	public	-	Returns area
getBasePrice()	double	public	-	Returns base price
calculatePrice()	double	public abstract	-	Calculates price
calculatePrice()	double	public abstract	double discountPct	Applies discount
displayAssetInfo()	void	public abstract	-	Prints property info

ResidentialProperty Methods

Method Name	Return Type	Access	Parameters	Work Done
calculatePrice()	double	public	-	Adds bedroom premium
calculatePrice()	double	public	double discountPct	Applies discount
generateListing()	void	public	-	Prints listing card
displayAssetInfo()	void	public	-	Prints residential info

LuxuryResidence Methods

Method Name	Return Type	Access	Parameters	Work Done
calculatePrice()	double	public	-	Adds luxury charges
calculatePrice()	double	public	double discountPct	Applies discount
generateListing()	void	public	-	Prints luxury card
displayAssetInfo()	void	public	-	Prints luxury info

PropertyAgent Methods

Method Name	Return Type	Access	Parameters	Work Done
getAgentName()	String	public	-	Returns agent name
calculateCommission()	double	public	double salePrice	Calculates commission
displayAgentInfo()	void	public	-	Prints agent info

13.2.4 Constructors

Class	Constructor Parameters
Asset	String assetId, String ownerName, String registrationDate
Asset	String assetId, String ownerName
Property	assetId, ownerName, registrationDate, location, areaSqFt, basePrice
ResidentialProperty	assetId, ownerName, registrationDate, location, areaSqFt, basePrice, bedrooms, bathrooms, hasParking
LuxuryResidence	assetId, ownerName, registrationDate, location, areaSqFt, basePrice, bedrooms, bathrooms, hasParking, hasPool, hasGym, luxuryMarkupPercent
PropertyAgent	String agentId, String agentName, double commissionRate

13.3 Expected Output

```

=== Residential Property ===
Asset ID : A001 | Owner: Hasan | Reg: 2024-01-10
Location : Sylhet City | Area: 1200.0 sqft

```

Bedrooms : 3 Bathrooms: 2 Parking: YES
*** RESIDENTIAL LISTING ***
[A001] 3BHK at Sylhet City 1200 sqft Parking: YES
Sale Price : 3650000.0
With 5% Disc : 3467500.0
=== Luxury Residence ===
Asset ID : A002 Owner: Mina Location: Gulshan
Area: 3500.0 sqft Beds: 5 Baths: 4
Pool: YES Gym: YES Markup: 25.0%
*** LUXURY LISTING ***
[A002] LUXURY 5BHK at Gulshan Pool: YES Gym: YES
Luxury Price : 15700000.0
=== Agent Info ===
ID: AG01 Name: Sabbir Commission: 2.5%
Agent Comm : 392500.0
Total Listings: 2

13.4 Main Function Code

01.	public class RealEstateSystem {
02.	public static void main(String[] args) {
03.	
04.	PropertyAgent agent = new PropertyAgent("AG01", "Sabbir", 2.5);
05.	
06.	ResidentialProperty rp = new ResidentialProperty(
07.	"A001", "Hasan", "2024-01-10",
08.	"Sylhet City", 1200.0, 3500000.0,
09.	3, 2, true
10.	);
11.	
12.	LuxuryResidence lr = new LuxuryResidence(
13.	"A002", "Mina", "2024-03-20",
14.	"Gulshan", 3500.0, 12000000.0,
15.	5, 4, true, true, true, 25.0
16.	);
17.	
18.	rp.displayAssetInfo();
19.	rp.generateListing();
20.	
21.	System.out.println("Sale Price : " + rp.calculatePrice());
22.	System.out.println("With 5% Disc : " + rp.calculatePrice(5));

```
23.
24.     lr.displayAssetInfo();
25.     lr.generateListing();
26.
27.     System.out.println("Luxury Price : " + lr.calculatePrice());
28.
29.     agent.displayAgentInfo();
30.     System.out.println("Agent Comm : " +
31.         agent.calculateCommission(lr.calculatePrice()));
32.
33.     System.out.println("Total Listings: " + Property.totalListings);
34. }
```

Problem 14 : Telecommunication Subscriber Billing System

14.1 Problem Statement

ConnectBD Telecom needs a software system to manage different types of subscribers, such as Postpaid Subscribers and Premium Postpaid Subscribers.

Every subscriber has a subscriber ID, subscriber name, and email.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Invoiceable` declares a method `generateInvoice()` that must be implemented by invoiceable subscribers.

The abstract class `Subscriber` stores common subscriber information.

The abstract class `MobileSubscriber` extends `Subscriber` and adds phone number and network type.

`PostpaidSubscriber` extends `MobileSubscriber` and implements `Invoiceable`. It adds monthly bill limit, data usage, and extra data charge rate.

`PremiumPostpaid` extends `PostpaidSubscriber` and adds manager name and rollover data.

A standalone class `SimCard` stores SIM card information such as SIM number, operator name, and activation date.

A static variable `totalSubscribers` inside `MobileSubscriber` tracks the total number of mobile subscriber objects created.

The class hierarchy is:

```
Subscriber -> MobileSubscriber -> PostpaidSubscriber -> PremiumPostpaid
```

```
PostpaidSubscriber implements Invoiceable
```

The system must support:

- displaying SIM card information
- displaying subscriber profiles
- calculating postpaid bill
- calculating bill with extra data using method overloading
- applying rollover data for premium subscribers
- generating invoices
- tracking total subscribers using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

14.2 Solving Guidelines

14.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Invoiceable	interface	-
Subscriber	abstract class	-
MobileSubscriber	abstract class	extends Subscriber
PostpaidSubscriber	concrete class	extends MobileSubscriber, implements Invoiceable
PremiumPostpaid	concrete class	extends PostpaidSubscriber
SimCard	standalone class	-
TelecomSystem	Driver Class(main)	-

14.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Subscriber	subscriberId	String	Instance	private
Subscriber	subscriberName	String	Instance	private
Subscriber	email	String	Instance	private
MobileSubscriber	phoneNumber	String	Instance	private
MobileSubscriber	networkType	String	Instance	private
MobileSubscriber	totalSubscribers	int	Static	public
PostpaidSubscriber	monthlyLimit	double	Instance	private
PostpaidSubscriber	dataUsedGB	double	Instance	private
PostpaidSubscriber	ratePerExtraGB	double	Instance	private
PremiumPostpaid	managerName	String	Instance	private
PremiumPostpaid	rolloverDataGB	double	Instance	private
SimCard	simNumber	String	Instance	private
SimCard	operator	String	Instance	private
SimCard	activationDate	String	Instance	private

14.2.3 Methods

Invoiceable Method

Method Name	Return Type	Access	Parameters	Work Done
generateInvoice()	void	public abstract	-	Prints invoice

Subscriber Methods

Method Name	Return Type	Access	Parameters	Work Done
getSubscriberId()	String	public	-	Returns subscriber ID
getSubscriberName()	String	public	-	Returns subscriber name
getEmail()	String	public	-	Returns email
displayProfile()	void	public abstract	-	Prints profile

MobileSubscriber Methods

Method Name	Return Type	Access	Parameters	Work Done
getPhoneNumber()	String	public	-	Returns phone number
getNetworkType()	String	public	-	Returns network type
displayProfile()	void	public abstract	-	Prints mobile profile

PostpaidSubscriber Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateBill()	double	public	-	Returns base bill
calculateBill()	double	public	double extraGB	Adds extra data charge
generateInvoice()	void	public	-	Prints postpaid invoice
displayProfile()	void	public	-	Prints postpaid profile

PremiumPostpaid Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateBill()	double	public	-	Returns premium bill
calculateBill()	double	public	double extraGB	Applies rollover data
generateInvoice()	void	public	-	Prints premium invoice
displayProfile()	void	public	-	Prints premium profile

SimCard Methods

Method Name	Return Type	Access	Parameters	Work Done
getSimNumber()	String	public	-	Returns SIM number
displaySimInfo()	void	public	-	Prints SIM info

14.2.4 Constructors

Class	Constructor Parameters
Subscriber	String subscriberId, String subscriberName, String email
Subscriber	String subscriberId, String subscriberName
MobileSubscriber	subscriberId, subscriberName, email, phoneNumber, networkType
PostpaidSubscriber	subscriberId, subscriberName, email, phoneNumber, networkType, monthlyLimit, ratePerExtraGB
PremiumPostpaid	subscriberId, subscriberName, email, phoneNumber, networkType, monthlyLimit, ratePerExtraGB, managerName, rolloverDataGB
SimCard	String simNumber, String operator, String activationDate

14.3 Expected Output

```

=== SIM Card Info ===
SIM : 8801911XXXXXX | Operator: ConnectBD | Activated: 2023-06-01

=== Postpaid Subscriber ===
ID    : S001 | Name: Ruhul | Email: ruhul@mail.com
Phone : 01911000001 | Network: 4G
Limit : 500.0 | Rate/ExtraGB: 30.0

```

Bill (8GB used) : 740.0

--- POSTPAID INVOICE ---

Subscriber: Ruhul | Phone: 01911000001

Monthly Limit: 500.0 | Extra Charge: 240.0 | Total: 740.0

=== Premium Postpaid ===

ID : S002 | Name: Farida | Network: 5G

Limit : 800.0 | Rollover: 3.0 GB

Manager : Mr. Alam

Premium Bill : 800.0

--- PREMIUM INVOICE ---

Subscriber: Farida | Manager: Mr. Alam

Base: 800.0 | Rollover Applied: 3.0 GB | Total: 800.0

Total Subscribers: 2

14.4 Main Function Code

```
01. public class TelecomSystem {
02.     public static void main(String[] args) {
03.
04.         SimCard sim = new SimCard(
05.             "8801911XXXXX", "ConnectBD", "2023-06-01"
06.         );
07.
08.         PostpaidSubscriber ps = new PostpaidSubscriber(
09.             "S001", "Ruhul", "ruhul@mail.com",
10.             "01911000001", "4G", 500.0, 30.0
11.         );
12.
13.         PremiumPostpaid pp = new PremiumPostpaid(
14.             "S002", "Farida", "farida@mail.com",
15.             "01922000002", "5G", 800.0, 25.0,
16.             "Mr. Alam", 3.0
17.         );
18.
19.         sim.displaySimInfo();
20.
21.         ps.displayProfile();
22.         System.out.println("Bill (8GB used) : " + ps.calculateBill(8));
23.         ps.generateInvoice();
24.
25.         pp.displayProfile();
26.         System.out.println("Premium Bill : " + pp.calculateBill());
```

```
27.         pp.generateInvoice();
28.
29.         System.out.println("Total Subscribers: " +
                               MobileSubscriber.totalSubscribers);
30.     }
31. }
```

Problem 15 : University Staff Payroll & Publication System

15.1 Problem Statement

SUST needs a software system to manage different types of staff, such as Academic Staff and Senior Professors.

Every person has a person ID, full name, and date of birth.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Publishable` declares a method `publishResearch()` that must be implemented by publishable academic staff.

The abstract class `Person` stores common personal information.

The abstract class `Staff` extends `Person` and adds designation, basic salary, and joining year.

`AcademicStaff` extends `Staff` and implements `Publishable`. It adds publication count, research grant amount, and specialization.

`SeniorProfessor` extends `AcademicStaff` and adds number of PhD students supervised and conference bonus.

A standalone class `Department` stores department information such as department code, department name, and head of department.

A static variable `totalStaff` inside `Staff` tracks the total number of staff objects created.

The class hierarchy is:

```
Person -> Staff -> AcademicStaff -> SeniorProfessor
```

```
AcademicStaff implements Publishable
```

The system must support:

- displaying department information
- displaying staff information
- calculating monthly compensation
- calculating multi-month compensation using method overloading
- applying research grant and conference bonus
- publishing research summary
- tracking total staff using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

15.2 Solving Guidelines

15.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Publishable	interface	-
Person	abstract class	-
Staff	abstract class	extends Person
AcademicStaff	concrete class	extends Staff, implements Publishable
SeniorProfessor	concrete class	extends AcademicStaff
Department	standalone class	-
StaffPayrollSystem	Driver Class(main)	-

15.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Person	personId	String	Instance	private
Person	fullName	String	Instance	private
Person	dateOfBirth	String	Instance	private
Staff	designation	String	Instance	private
Staff	basicSalary	double	Instance	private
Staff	joiningYear	int	Instance	private
Staff	totalStaff	int	Static	public
AcademicStaff	publicationsCount	int	Instance	private
AcademicStaff	researchGrantAmount	double	Instance	private
AcademicStaff	specialization	String	Instance	private
SeniorProfessor	phdStudentsSupervised	int	Instance	private
SeniorProfessor	conferenceBonus	double	Instance	private
Department	deptCode	String	Instance	private
Department	deptName	String	Instance	private
Department	hod	String	Instance	private

15.2.3 Methods

Publishable Method

Method Name	Return Type	Access	Parameters	Work Done
publishResearch()	void	public abstract	-	Prints research info

Person Methods

Method Name	Return Type	Access	Parameters	Work Done
getPersonId()	String	public	-	Returns person ID
getFullName()	String	public	-	Returns full name
getDateOfBirth()	String	public	-	Returns date of birth
displayInfo()	void	public abstract	-	Prints person info

Staff Methods

Method Name	Return Type	Access	Parameters	Work Done
getDesignation()	String	public	-	Returns designation
getBasicSalary()	double	public	-	Returns basic salary
getJoiningYear()	int	public	-	Returns joining year
calculateCompensation()	double	public abstract	-	Calculates monthly pay
calculateCompensation()	double	public abstract	int months	Calculates multi-month pay
displayInfo()	void	public abstract	-	Prints staff info

AcademicStaff Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateCompensation()	double	public	-	Adds research grant
calculateCompensation()	double	public	int months	Calculates multi-month
publishResearch()	void	public	-	Prints research summary
displayInfo()	void	public	-	Prints academic info

SeniorProfessor Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateCompensation()	double	public	-	Adds conference bonus
calculateCompensation()	double	public	int months	Calculates multi-month
publishResearch()	void	public	-	Prints senior research
displayInfo()	void	public	-	Prints professor info

Department Methods

Method Name	Return Type	Access	Parameters	Work Done
getDeptName()	String	public	-	Returns dept name
displayDeptInfo()	void	public	-	Prints dept info

15.2.4 Constructors

Class	Constructor Parameters
Person	String personId, String fullName, String dateOfBirth
Person	String personId, String fullName
Staff	personId, fullName, dateOfBirth, designation, basicSalary, joiningYear
AcademicStaff	personId, fullName, dateOfBirth, designation, basicSalary, joiningYear, publicationsCount, researchGrantAmount, specialization
SeniorProfessor	personId, fullName, dateOfBirth, designation, basicSalary, joiningYear, publicationsCount, researchGrantAmount, specialization, phdStudentsSupervised, conferenceBonus
Department	String deptCode, String deptName, String hod

15.3 Expected Output

```
=== Department Info ===
```

```
Code: CSE | Name: Computer Science | HOD: Prof. Karim
```

=== Academic Staff ===

ID: ST01 | Name: Dr. Nusrat | DOB: 1985-04-12

Designation: Associate Prof | Salary: 75000.0

Joined: 2015 | Specialization: ML | Publications: 18

Research Grant: 15000.0

Monthly Comp : 90000.0

6-Month Comp : 540000.0

Published 18 research papers in ML.

=== Senior Professor ===

ID: ST02 | Name: Prof. Jamal | Designation: Professor

Salary: 120000.0 | Specialization: AI | Publications: 45

PhD Students Supervised: 8 | Conference Bonus: 20000.0

Senior Monthly: 170000.0

Published 45 papers in AI. Supervising 8 PhD students.

Total Staff: 2

15.4 Main Function Code

```
01. public class StaffPayrollSystem {
02.     public static void main(String[] args) {
03.
04.         Department dept = new Department("CSE", "Computer Science", "Prof. Karim");
05.
06.         AcademicStaff as = new AcademicStaff(
07.             "ST01", "Dr. Nusrat", "1985-04-12",
08.             "Associate Prof", 75000.0, 2015,
09.             18, 15000.0, "ML"
10.         );
11.
12.         SeniorProfessor sp = new SeniorProfessor(
13.             "ST02", "Prof. Jamal", "1970-09-01",
14.             "Professor", 120000.0, 2005,
15.             45, 30000.0, "AI", 8, 20000.0
16.         );
17.
18.         dept.displayDeptInfo();
19.
20.         as.displayInfo();
21.         System.out.println("Monthly Comp : " + as.calculateCompensation());
22.         System.out.println("6-Month Comp : " + as.calculateCompensation(6));
23.         as.publishResearch();
24.     }
```

```
25.      sp.displayInfo();
26.      System.out.println("Senior Monthly: " + sp.calculateCompensation());
27.      sp.publishResearch();
28.
29.      System.out.println("Total Staff: " + Staff.totalStaff);
30.  }
31. }
```

Problem 16 : Online Gaming Platform Ranking System

16.1 Problem Statement

ArenaPro Gaming needs a software system to manage different types of players, such as Ranked Players and Pro Players.

Every user has a user ID, username, and email.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface Rankable declares a method `updateRank()` that must be implemented by rankable players.

The abstract class User stores common user information.

The abstract class GamePlayer extends User and adds games played and wins.

RankedPlayer extends GamePlayer and implements Rankable. It adds rank tier, rank points, and season wins.

ProPlayer extends RankedPlayer and adds sponsor name, tournament wins, and prize money earned.

A standalone class GameTitle stores game information such as game ID, game name, and genre.

A static variable `totalUsers` inside GamePlayer tracks the total number of game player objects created.

The class hierarchy is:

```
User -> GamePlayer -> RankedPlayer -> ProPlayer
```

```
RankedPlayer implements Rankable
```

The system must support:

- displaying game title information
- displaying player profile cards
- calculating win rate
- calculating XP for match results
- calculating XP with kill bonus using method overloading
- updating rank using rank points
- tracking total users using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

16.2 Solving Guidelines

16.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Rankable	interface	-
User	abstract class	-
GamePlayer	abstract class	extends User
RankedPlayer	concrete class	extends GamePlayer, implements Rankable
ProPlayer	concrete class	extends RankedPlayer
GameTitle	standalone class	-
GamingPlatform	Driver Class(main)	-

16.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
User	userId	String	Instance	private
User	username	String	Instance	private
User	email	String	Instance	private
GamePlayer	gamesPlayed	int	Instance	private
GamePlayer	wins	int	Instance	private
GamePlayer	totalUsers	int	Static	public
RankedPlayer	rankTier	String	Instance	private
RankedPlayer	rankPoints	int	Instance	private
RankedPlayer	seasonWins	int	Instance	private
ProPlayer	sponsorName	String	Instance	private
ProPlayer	tournamentWins	int	Instance	private
ProPlayer	prizeMoneyEarned	double	Instance	private
GameTitle	gameId	String	Instance	private
GameTitle	gameName	String	Instance	private
GameTitle	genre	String	Instance	private

16.2.3 Methods

Rankable Method

Method Name	Return Type	Access	Parameters	Work Done
updateRank()	void	public abstract	-	Updates rank tier

User Methods

Method Name	Return Type	Access	Parameters	Work Done
getUserId()	String	public	-	Returns user ID
getUsername()	String	public	-	Returns username
getEmail()	String	public	-	Returns email
displayProfile()	void	public abstract	-	Prints user profile

GamePlayer Methods

Method Name	Return Type	Access	Parameters	Work Done
getGamesPlayed()	int	public	-	Returns games count
getWins()	int	public	-	Returns win count
getWinRate()	double	public	-	Calculates win rate
calculateXP()	int	public abstract	boolean isWin	Calculates XP
calculateXP()	int	public abstract	boolean isWin, int kills	Adds kill bonus
displayProfile()	void	public abstract	-	Prints player card

RankedPlayer Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateXP()	int	public	boolean isWin	Calculates match XP
calculateXP()	int	public	boolean isWin, int kills	Adds kill bonus
updateRank()	void	public	-	Updates rank tier
displayProfile()	void	public	-	Prints ranked card

ProPlayer Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateXP()	int	public	boolean isWin	Applies pro bonus
calculateXP()	int	public	boolean isWin, int kills	Adds kill bonus
updateRank()	void	public	-	Updates pro rank
displayProfile()	void	public	-	Prints pro card

GameTitle Methods

Method Name	Return Type	Access	Parameters	Work Done
getGameName()	String	public	-	Returns game name
displayGameInfo()	void	public	-	Prints game info

16.2.4 Constructors

Class	Constructor Parameters
User	String userId, String username, String email
User	String userId, String username
GamePlayer	userId, username, email, gamesPlayed, wins
RankedPlayer	userId, username, email, gamesPlayed, wins, rankTier, rankPoints
ProPlayer	userId, username, email, gamesPlayed, wins, rankTier, rankPoints, sponsorName, tournamentWins, prizeMoneyEarned
GameTitle	String gameId, String gameName, String genre

16.3 Expected Output

```

=== Game Title ===
ID: G001 | Name: ValorStrike | Genre: FPS

=== Ranked Player Card ===
User: ShadowX | Email: shadow@arena.com
Games: 120 | Wins: 80 | Win Rate: 66.67%
Tier: Gold | Rank Points: 1500

XP (win)          : 200
XP (win,15 kills): 350
Rank Updated: ShadowX is now PLATINUM (1700 pts)

=== Pro Player Card ===
User: NightHawk | Games: 500 | Wins: 420
Tier: Diamond | Rank Points: 4200

```

Sponsor: RedBull | Tournament Wins: 12

Prize Money: 850000.0

Pro XP (win): 300

Pro Rank Updated: NightHawk remains LEGENDARY (4500 pts, 12 tournament wins)

Total Users: 2

16.4 Main Function Code

```
01. public class GamingPlatform {
02.     public static void main(String[] args) {
03.
04.         GameTitle gt = new GameTitle("G001", "ValorStrike", "FPS");
05.
06.         RankedPlayer rp = new RankedPlayer(
07.             "U001", "ShadowX", "shadow@arena.com",
08.             120, 80, "Gold", 1500
09.         );
10.
11.         ProPlayer pp = new ProPlayer(
12.             "U002", "NightHawk", "hawk@arena.com",
13.             500, 420, "Diamond", 4200,
14.             "RedBull", 12, 850000.0
15.         );
16.
17.         gt.displayGameInfo();
18.
19.         rp.displayProfile();
20.         System.out.println("XP (win)           : " + rp.calculateXP(true));
21.         System.out.println("XP (win,15 kills): " + rp.calculateXP(true, 15));
22.         rp.updateRank();
23.
24.         pp.displayProfile();
25.         System.out.println("Pro XP (win): " + pp.calculateXP(true));
26.         pp.updateRank();
27.
28.         System.out.println("Total Users: " + GamePlayer.totalUsers);
29.     }
30. }
```

Problem 17 : Warehouse Inventory and Supply Chain System

17.1 Problem Statement

LogiStore Warehousing needs a software system to manage different types of inventory items, such as Perishable Items and Cold Chain Items.

Every item has an item ID, item name, and unit of measure.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Trackable` declares a method `generateStockReport()` that must be implemented by trackable inventory items.

The abstract class `Item` stores common item information.

The abstract class `InventoryItem` extends `Item` and adds quantity, unit cost, and supplier name.

`PerishableItem` extends `InventoryItem` and implements `Trackable`. It adds expiry date, shelf life in days, and expiry status.

`ColdChainItem` extends `PerishableItem` and adds required temperature and cold storage cost per day.

A standalone class `Warehouse` stores warehouse information such as warehouse ID, location, and capacity in tons.

A static variable `totalItemTypes` inside `InventoryItem` tracks the total number of inventory item objects created.

The class hierarchy is:

```
Item -> InventoryItem -> PerishableItem -> ColdChainItem
```

```
PerishableItem implements Trackable
```

The system must support:

- displaying warehouse information
- displaying inventory item information
- calculating stock value
- calculating stock value with storage cost using method overloading
- checking expiry status
- generating stock reports
- tracking total item types using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

17.2 Solving Guidelines

17.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Trackable	interface	-
Item	abstract class	-
InventoryItem	abstract class	extends Item
PerishableItem	concrete class	extends InventoryItem, implements Trackable
ColdChainItem	concrete class	extends PerishableItem
Warehouse	standalone class	-
InventorySystem	Driver Class(main)	-

17.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Item	itemId	String	Instance	private
Item	itemName	String	Instance	private
Item	unitOfMeasure	String	Instance	private
InventoryItem	quantity	int	Instance	private
InventoryItem	unitCost	double	Instance	private
InventoryItem	supplierName	String	Instance	private
InventoryItem	totalItemTypes	int	Static	public
PerishableItem	expiryDate	String	Instance	private
PerishableItem	shelfLifeDays	int	Instance	private
PerishableItem	isExpired	boolean	Instance	private
ColdChainItem	requiredTempC	double	Instance	private
ColdChainItem	coldStorageCostPerDay	double	Instance	private
Warehouse	warehouseId	String	Instance	private
Warehouse	location	String	Instance	private
Warehouse	capacityTons	double	Instance	private

17.2.3 Methods

Trackable Method

Method Name	Return Type	Access	Parameters	Work Done
generateStockReport()	void	public abstract	-	Prints stock report

Item Methods

Method Name	Return Type	Access	Parameters	Work Done
getItemId()	String	public	-	Returns item ID
getItemName()	String	public	-	Returns item name
getUnitOfMeasure()	String	public	-	Returns unit
displayItemInfo()	void	public abstract	-	Prints item info

InventoryItem Methods

Method Name	Return Type	Access	Parameters	Work Done
getQuantity()	int	public	-	Returns quantity
getUnitCost()	double	public	-	Returns unit cost
getSupplierName()	String	public	-	Returns supplier
calculateStockValue()	double	public abstract	-	Calculates stock value
calculateStockValue()	double	public abstract	int days	Adds storage cost
displayItemInfo()	void	public abstract	-	Prints inventory info

PerishableItem Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateStockValue()	double	public	-	Returns stock value
calculateStockValue()	double	public	int days	Checks shelf life
checkExpiry()	String	public	-	Returns expiry status
generateStockReport()	void	public	-	Prints perishable report
displayItemInfo()	void	public	-	Prints perishable details

ColdChainItem Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateStockValue()	double	public	-	Adds one-day cold cost
calculateStockValue()	double	public	int days	Adds cold storage cost
generateStockReport()	void	public	-	Prints cold report
displayItemInfo()	void	public	-	Prints cold item info

Warehouse Methods

Method Name	Return Type	Access	Parameters	Work Done
getLocation()	String	public	-	Returns location
displayWarehouseInfo()	void	public	-	Prints warehouse info

17.2.4 Constructors

Class	Constructor Parameters
Item	String itemId, String itemName, String unitOfMeasure
Item	String itemId, String itemName
InventoryItem	itemId, itemName, unitOfMeasure, quantity, unitCost, supplierName
PerishableItem	itemId, itemName, unitOfMeasure, quantity, unitCost, supplierName, expiryDate, shelfLifeDays
ColdChainItem	itemId, itemName, unitOfMeasure, quantity, unitCost, supplierName, expiryDate, shelfLifeDays, requiredTempC, coldStorageCostPerDay
Warehouse	String warehouseId, String location, double capacityTons

17.3 Expected Output

```

=== Warehouse Info ===
ID: WH01 | Location: Sylhet Industrial Zone | Capacity: 500.0 tons

```

```

=== Perishable Item ===
ID: I001 | Name: Fresh Mango | Unit: kg
Qty: 200 | Unit Cost: 80.0 | Supplier: AgroFresh
Expiry: 2025-07-15 | Shelf Life: 30 days

Expiry Status   : Not Expired
Stock Value     : 16000.0
Value (10 days): 16000.0

--- PERISHABLE STOCK REPORT ---
Item: Fresh Mango | Qty: 200 | Value: 16000.0 | Expires: 2025-07-15

=== Cold Chain Item ===
ID: I002 | Name: Frozen Fish | Unit: kg
Qty: 500 | Unit Cost: 150.0 | Supplier: SeaFarm
Expiry: 2025-12-01 | Required Temp: -18.0°C | Cold Cost/Day: 500.0

Cold Value      : 75500.0
Cold (30 days): 90000.0

--- COLD CHAIN STOCK REPORT ---
Item: Frozen Fish | Qty: 500 | Temp: -18.0°C | Cold Value: 75500.0

Total Item Types: 2

```

17.4 Main Function Code

```

01. public class InventorySystem {
02.     public static void main(String[] args) {
03.
04.         Warehouse wh = new Warehouse(
05.             "WH01", "Sylhet Industrial Zone", 500.0
06.         );
07.
08.         PerishableItem pi = new PerishableItem(
09.             "I001", "Fresh Mango", "kg",
10.             200, 80.0, "AgroFresh",
11.             "2025-07-15", 30
12.         );
13.
14.         ColdChainItem ci = new ColdChainItem(
15.             "I002", "Frozen Fish", "kg",
16.             500, 150.0, "SeaFarm",
17.             "2025-12-01", 90, -18.0, 500.0
18.         );
19.
20.         wh.displayWarehouseInfo();
21.

```

```
22.         pi.displayItemInfo();
23.         System.out.println("Expiry Status  : " + pi.checkExpiry());
24.         System.out.println("Stock Value    : " + pi.calculateStockValue());
25.         System.out.println("Value (10 days): " + pi.calculateStockValue(10));
26.         pi.generateStockReport();
27.
28.         ci.displayItemInfo();
29.         System.out.println("Cold Value     : " + ci.calculateStockValue());
30.         System.out.println("Cold (30 days): " + ci.calculateStockValue(30));
31.         ci.generateStockReport();
32.
33.         System.out.println("Total Item Types: " + InventoryItem.totalItemTypes);
34.     }
35. }
```

Problem 18 : Media Streaming Platform Subscription System

18.1 Problem Statement

StreamZone needs a software system to manage different types of streaming accounts, such as Paid Accounts and Family Plan Accounts.

Every account has an account ID, owner email, and password.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Streamable` declares a method `startStream()` that must be implemented by streamable paid accounts.

The abstract class `Account` stores common account information.

The abstract class `StreamingAccount` extends `Account` and adds watched hours and data limit in GB.

`PaidAccount` extends `StreamingAccount` and implements `Streamable`. It adds plan name, monthly fee, and HD access status.

`FamilyPlanAccount` extends `PaidAccount` and adds maximum profiles, active profiles, and shared data pool.

A standalone class `ContentItem` stores content information such as content ID, title, and duration in minutes.

A static variable `totalAccounts` inside `StreamingAccount` tracks the total number of streaming account objects created.

The class hierarchy is:

```
Account -> StreamingAccount -> PaidAccount -> FamilyPlanAccount
```

```
PaidAccount implements Streamable
```

The system must support:

- displaying content information
- displaying account information
- calculating monthly charge
- calculating charge with extra data using method overloading
- starting stream using interface implementation
- tracking total accounts using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

18.2 Solving Guidelines

18.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Streamable	interface	-
Account	abstract class	-
StreamingAccount	abstract class	extends Account
PaidAccount	concrete class	extends StreamingAccount, implements Streamable
FamilyPlanAccount	concrete class	extends PaidAccount
ContentItem	standalone class	-
StreamingSystem	Driver Class(main)	-

18.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Account	accountId	String	Instance	private
Account	ownerEmail	String	Instance	private
Account	password	String	Instance	private
StreamingAccount	watchedHours	double	Instance	private
StreamingAccount	dataLimitGB	double	Instance	private
StreamingAccount	totalAccounts	int	Static	public
PaidAccount	planName	String	Instance	private
PaidAccount	monthlyFee	double	Instance	private
PaidAccount	isHDEnabled	boolean	Instance	private
FamilyPlanAccount	maxProfiles	int	Instance	private
FamilyPlanAccount	activeProfiles	int	Instance	private
FamilyPlanAccount	sharedDataPoolGB	double	Instance	private
ContentItem	contentId	String	Instance	private
ContentItem	title	String	Instance	private
ContentItem	durationMins	int	Instance	private

18.2.3 Methods

Streamable Method

Method Name	Return Type	Access	Parameters	Work Done
startStream()	void	public abstract	-	Starts streaming

Account Methods

Method Name	Return Type	Access	Parameters	Work Done
getAccountId()	String	public	-	Returns account ID
getOwnerEmail()	String	public	-	Returns email
displayAccountInfo()	void	public abstract	-	Prints account info

StreamingAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
getWatchedHours()	double	public	-	Returns watch hours
getDataLimitGB()	double	public	-	Returns data limit
addWatchTime()	void	public	double hrs	Adds watch time
calculateMonthlyCharge()	double	public abstract	-	Calculates charge
calculateMonthlyCharge()	double	public abstract	int extraDataGB	Adds extra data cost
displayAccountInfo()	void	public abstract	-	Prints account info

PaidAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateMonthlyCharge()	double	public	-	Returns monthly fee
calculateMonthlyCharge()	double	public	int extraDataGB	Adds extra data cost
startStream()	void	public	-	Starts paid stream
displayAccountInfo()	void	public	-	Prints paid account

FamilyPlanAccount Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateMonthlyCharge()	double	public	-	Adds profile charge
calculateMonthlyCharge()	double	public	int extraDataGB	Adds extra data cost
startStream()	void	public	-	Starts family stream
displayAccountInfo()	void	public	-	Prints family info

ContentItem Methods

Method Name	Return Type	Access	Parameters	Work Done
getTitle()	String	public	-	Returns title
displayContentInfo()	void	public	-	Prints content info

18.2.4 Constructors

Class	Constructor Parameters
Account	String accountId, String ownerEmail, String password
Account	String accountId, String ownerEmail
StreamingAccount	accountId, ownerEmail, password, dataLimitGB
PaidAccount	accountId, ownerEmail, password, dataLimitGB, planName, monthlyFee, isHDEnabled
FamilyPlanAccount	accountId, ownerEmail, password, dataLimitGB, planName, monthlyFee, isHDEnabled, maxProfiles, sharedDataPoolGB
ContentItem	String contentId, String title, int durationMins

18.3 Expected Output

```

=== Content Info ===
ID: C001 | Title: Dragon Chronicles | Duration: 142 mins

=== Paid Account ===

```

```
Account ID : A001 | Email: rana@mail.com
Data Limit : 50.0 GB | Plan: Standard
Fee       : 499.0 | HD: ENABLED

Monthly Charge : 499.0
With 5GB extra : 749.0

Streaming started for rana@mail.com | HD: YES
```

```
=== Family Plan Account ===
Account ID      : A002 | Email: mitu@mail.com
Plan           : Family | Fee: 999.0
Max Profiles   : 5 | Active: 1
Shared Pool    : 200.0 GB

Family Charge   : 1099.0

Family streaming started | Active Profiles: 1/5
Total Accounts: 2
```

18.4 Main Function Code

```
01. public class StreamingSystem {
02.     public static void main(String[] args) {
03.
04.         ContentItem ci = new ContentItem(
05.             "C001", "Dragon Chronicles", 142
06.         );
07.
08.         PaidAccount pa = new PaidAccount(
09.             "A001", "rana@mail.com", "pass123",
10.             50.0, "Standard", 499.0, true
11.         );
12.
13.         FamilyPlanAccount fa = new FamilyPlanAccount(
14.             "A002", "mitu@mail.com", "pass456",
15.             200.0, "Family", 999.0, true, 5, 200.0
16.         );
17.
18.         ci.displayContentInfo();
19.
20.         pa.displayAccountInfo();
21.         System.out.println("Monthly Charge : " + pa.calculateMonthlyCharge());
22.         System.out.println("With 5GB extra : " + pa.calculateMonthlyCharge(5));
23.         pa.startStream();
24.
25.         fa.displayAccountInfo();
26.         System.out.println("Family Charge : " + fa.calculateMonthlyCharge());
```

```
27.         fa.startStream();
28.
29.         System.out.println("Total Accounts: " + StreamingAccount.totalAccounts);
30.     }
31. }
```

Problem 19 : Agricultural Farm Management & Yield System

19.1 Problem Statement

GreenAcre Farms needs a software system to manage different types of crops, such as Cash Crops and Export Crops.

Every farm asset has an asset ID, farm name, and owner name.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Harvestable` declares a method `generateHarvestReport()` that must be implemented by harvestable crop classes.

The abstract class `FarmAsset` stores common farm asset information.

The abstract class `Crop` extends `FarmAsset` and adds crop name, planting date, and area in acres.

`CashCrop` extends `Crop` and implements `Harvestable`. It adds market price per kilogram, expected yield per acre, and harvest date.

`ExportCrop` extends `CashCrop` and adds export destination, certification label, and export premium percentage.

A standalone class `FarmPlot` stores plot information such as plot ID, soil type, and irrigation type.

A static variable `totalCropTypes` inside `Crop` tracks the total number of crop objects created.

The class hierarchy is:

```
FarmAsset -> Crop -> CashCrop -> ExportCrop
```

```
CashCrop implements Harvestable
```

The system must support:

- displaying farm plot information
- displaying crop asset information
- calculating expected revenue
- calculating actual revenue using method overloading
- adding export premium for export crops
- generating harvest reports
- tracking total crop types using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

19.2 Solving Guidelines

19.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Harvestable	interface	-
FarmAsset	abstract class	-
Crop	abstract class	extends FarmAsset
CashCrop	concrete class	extends Crop, implements Harvestable
ExportCrop	concrete class	extends CashCrop
FarmPlot	standalone class	-
FarmManagementSystem	Driver Class(main)	-

19.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
FarmAsset	assetId	String	Instance	private
FarmAsset	farmName	String	Instance	private
FarmAsset	ownerName	String	Instance	private
Crop	cropName	String	Instance	private
Crop	plantingDate	String	Instance	private
Crop	areaAcres	double	Instance	private
Crop	totalCropTypes	int	Static	public
CashCrop	marketPricePerKg	double	Instance	private
CashCrop	expectedYieldKgPerAcre	double	Instance	private
CashCrop	harvestDate	String	Instance	private
ExportCrop	exportDestination	String	Instance	private
ExportCrop	certificationLabel	String	Instance	private
ExportCrop	exportPremiumPercent	double	Instance	private
FarmPlot	plotId	String	Instance	private
FarmPlot	soilType	String	Instance	private
FarmPlot	irrigationType	String	Instance	private

19.2.3 Methods

Harvestable Method

Method Name	Return Type	Access	Parameters	Work Done
generateHarvestReport()	void	public abstract	-	Prints harvest report

FarmAsset Methods

Method Name	Return Type	Access	Parameters	Work Done
getAssetId()	String	public	-	Returns asset ID
getFarmName()	String	public	-	Returns farm name
getOwnerName()	String	public	-	Returns owner name
displayAssetInfo()	void	public abstract	-	Prints asset info

Crop Methods

Method Name	Return Type	Access	Parameters	Work Done
getCropName()	String	public	-	Returns crop name
getAreaAcres()	double	public	-	Returns area
calculateRevenue()	double	public abstract	-	Calculates revenue
calculateRevenue()	double	public abstract	double actualYieldKg	Calculates actual revenue
displayAssetInfo()	void	public abstract	-	Prints crop info

CashCrop Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateRevenue()	double	public	-	Calculates expected revenue
calculateRevenue()	double	public	double actualYieldKg	Calculates actual revenue
generateHarvestReport()	void	public	-	Prints harvest report
displayAssetInfo()	void	public	-	Prints cash crop info

ExportCrop Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateRevenue()	double	public	-	Adds export premium
calculateRevenue()	double	public	double actualYieldKg	Adds premium to actual revenue
generateHarvestReport()	void	public	-	Prints export report
displayAssetInfo()	void	public	-	Prints export crop info

FarmPlot Methods

Method Name	Return Type	Access	Parameters	Work Done
getPlotId()	String	public	-	Returns plot ID
displayPlotInfo()	void	public	-	Prints plot info

19.2.4 Constructors

Class	Constructor Parameters
FarmAsset	String assetId, String farmName, String ownerName
FarmAsset	String assetId, String farmName
Crop	assetId, farmName, ownerName, cropName, plantingDate, areaAcres
CashCrop	assetId, farmName, ownerName, cropName, plantingDate, areaAcres, marketPricePerKg, expectedYieldKgPerAcre, harvestDate
ExportCrop	assetId, farmName, ownerName, cropName, plantingDate, areaAcres, marketPricePerKg, expectedYieldKgPerAcre, harvestDate, exportDestination, certificationLabel, exportPremiumPercent
FarmPlot	String plotId, String soilType, String irrigationType

19.3 Expected Output

=== Farm Plot Info ===	
Plot ID	: PL01
Soil	: Alluvial
Irrigation: Drip Irrigation	
=== Cash Crop Details ===	
Asset ID	: FA01 Farm: GreenAcre Owner: Rahim
Crop	: Paddy Area: 10.0 acres Planted: 2025-01-15
Market Price: 30.0/kg Expected Yield: 800.0 kg/acre Harvest: 2025-06-01	
Expected Revenue: 240000.0	
Actual (7500kg) : 225000.0	
--- HARVEST REPORT: Paddy ---	
Total Expected: 8000.0 kg Revenue: 240000.0	
=== Export Crop Details ===	
Asset ID	: FA02 Crop: Hilsa Fish
Area: 5.0 acres Price: 1200.0/kg Dest: Japan	
Certification: HACCP Export Premium: 20.0%	
Export Revenue : 288000.0	
--- EXPORT HARVEST REPORT: Hilsa Fish ---	
Destination: Japan Revenue: 240000.0 Premium: 48000.0 Total: 288000.0	
Total Crop Types: 2	

19.4 Main Function Code

01.	public class FarmManagementSystem {
02.	public static void main(String[] args) {
03.	
04.	FarmPlot fp = new FarmPlot(
05.	"PL01", "Alluvial", "Drip Irrigation"
06.	);
07.	
08.	CashCrop cc = new CashCrop(
09.	"FA01", "GreenAcre", "Rahim",
10.	"Paddy", "2025-01-15",
11.	10.0, 30.0, 800.0,
12.	"2025-06-01"
13.	);
14.	
15.	ExportCrop ec = new ExportCrop(
16.	"FA02", "GreenAcre", "Rahim",
17.	"Hilsa Fish", "2025-02-01",

```
18.         5.0, 1200.0, 200.0,
19.         "2025-07-01", "Japan", "HACCP", 20.0
20.     );
21.
22.     fp.displayPlotInfo();
23.
24.     cc.displayAssetInfo();
25.     System.out.println("Expected Revenue: " + cc.calculateRevenue());
26.     System.out.println("Actual (7500kg) : " + cc.calculateRevenue(7500));
27.     cc.generateHarvestReport();
28.
29.     ec.displayAssetInfo();
30.     System.out.println("Export Revenue  : " + ec.calculateRevenue());
31.     ec.generateHarvestReport();
32.
33.     System.out.println("Total Crop Types: " + Crop.totalCropTypes);
34. }
35. }
```

Problem 20 : Power Utility Energy Billing System

20.1 Problem Statement

GridPower Utility needs a software system to manage different types of electricity consumers, such as Industrial Consumers and Heavy Industrial Consumers.

Every consumer has a consumer ID, consumer name, and address.

The system should be designed using abstraction, inheritance, interface implementation, constructor overloading, method overriding, method overloading, and static variable tracking.

The interface `Billable` declares a method `generateEnergyBill()` that must be implemented by billable energy consumers.

The abstract class `Consumer` stores common consumer information.

The abstract class `EnergyConsumer` extends `Consumer` and adds meter ID, consumed units in kWh, and tariff rate per unit.

`IndustrialConsumer` extends `EnergyConsumer` and implements `Billable`. It adds load factor percentage, peak demand in kW, and demand charge per kW.

`HeavyIndustrialConsumer` extends `IndustrialConsumer` and adds power factor penalty and backup generator capacity in kVA.

A standalone class `EnergyMeter` stores meter information such as meter serial number, meter type, and installation date.

A static variable `totalConsumers` inside `EnergyConsumer` tracks the total number of energy consumer objects created.

The class hierarchy is:

```
Consumer -> EnergyConsumer -> IndustrialConsumer -> HeavyIndustrialConsumer
```

```
IndustrialConsumer implements Billable
```

The system must support:

- displaying energy meter information
- displaying consumer information
- calculating industrial energy bill
- calculating bill with extra units using method overloading
- calculating heavy industrial bill with penalty and generator charge
- generating formatted energy bills
- tracking total consumers using a static variable

Follow the guidelines provided below to implement all necessary classes, interfaces, constructors, methods, and anything else required to produce the expected output.

20.2 Solving Guidelines

20.2.1 Class Hierarchy

Class Name	Type	Inherits / Implements
Billable	interface	-
Consumer	abstract class	-
EnergyConsumer	abstract class	extends Consumer
IndustrialConsumer	concrete class	extends EnergyConsumer, implements Billable
HeavyIndustrialConsumer	concrete class	extends IndustrialConsumer
EnergyMeter	standalone class	-
PowerUtilitySystem	Driver Class(main)	-

20.2.2 Variables

Class	Variable Name	Data Type	Var Type	Access
Consumer	consumerId	String	Instance	private
Consumer	consumerName	String	Instance	private
Consumer	address	String	Instance	private
EnergyConsumer	meterId	String	Instance	private
EnergyConsumer	unitsConsumedKWh	double	Instance	private
EnergyConsumer	tariffRatePerUnit	double	Instance	private
EnergyConsumer	totalConsumers	int	Static	public
IndustrialConsumer	loadFactorPercent	double	Instance	private
IndustrialConsumer	peakDemandKW	double	Instance	private
IndustrialConsumer	demandChargePerKW	double	Instance	private
HeavyIndustrialConsumer	powerFactorPenalty	double	Instance	private
HeavyIndustrialConsumer	backupGeneratorKVA	double	Instance	private
EnergyMeter	meterSerialNo	String	Instance	private
EnergyMeter	meterType	String	Instance	private
EnergyMeter	installationDate	String	Instance	private

20.2.3 Methods

Billable Method

Method Name	Return Type	Access	Parameters	Work Done
generateEnergyBill()	void	public abstract	-	Prints energy bill

Consumer Methods

Method Name	Return Type	Access	Parameters	Work Done
getConsumerId()	String	public	-	Returns consumer ID
getConsumerName()	String	public	-	Returns consumer name
getAddress()	String	public	-	Returns address
displayConsumerInfo()	void	public abstract	-	Prints consumer info

EnergyConsumer Methods

Method Name	Return Type	Access	Parameters	Work Done
getMeterId()	String	public	-	Returns meter ID
getUnitsConsumed()	double	public	-	Returns units
getTariffRate()	double	public	-	Returns tariff rate
calculateBill()	double	public abstract	-	Calculates bill
calculateBill()	double	public abstract	double extraUnits	Adds extra units
displayConsumerInfo()	void	public abstract	-	Prints energy info

IndustrialConsumer Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateBill()	double	public	-	Calculates base bill
calculateBill()	double	public	double extraUnits	Adds extra unit bill
generateEnergyBill()	void	public	-	Prints industrial bill
displayConsumerInfo()	void	public	-	Prints industrial info

HeavyIndustrialConsumer Methods

Method Name	Return Type	Access	Parameters	Work Done
calculateBill()	double	public	-	Adds heavy charges
calculateBill()	double	public	double extraUnits	Adds extra unit bill
generateEnergyBill()	void	public	-	Prints heavy bill
displayConsumerInfo()	void	public	-	Prints heavy info

EnergyMeter Methods

Method Name	Return Type	Access	Parameters	Work Done
getMeterSerialNo()	String	public	-	Returns serial no
displayMeterInfo()	void	public	-	Prints meter info

20.2.4 Constructors

Class	Constructor Parameters
Consumer	String consumerId, String consumerName, String address
Consumer	String consumerId, String consumerName
EnergyConsumer	consumerId, consumerName, address, meterId, unitsConsumedKWh, tariffRatePerUnit
IndustrialConsumer	consumerId, consumerName, address, meterId, unitsConsumedKWh, tariffRatePerUnit, loadFactorPercent, peakDemandKW, demandChargePerKW
HeavyIndustrialConsumer	consumerId, consumerName, address, meterId, unitsConsumedKWh, tariffRatePerUnit, loadFactorPercent, peakDemandKW, demandChargePerKW, powerFactorPenalty, backupGeneratorKVA
EnergyMeter	String meterSerialNo, String meterType, String installationDate

20.3 Expected Output

```
=== Energy Meter Info ===
```

Serial	: MT-9981 Type: Smart Meter Installed: 2022-08-10
=== Industrial Consumer ===	
ID: C001 Name: Apex Textiles Address: Gazipur	
Meter ID: MT-9981 Units: 12000.0 kWh Tariff: 8.5/unit	
Load Factor: 75.0% Peak Demand: 500.0 kW Demand Charge: 150.0/kW	
Industrial Bill : 177000.0	
With 1000 extra kWh: 185500.0	
===== INDUSTRIAL ENERGY BILL =====	
Consumer : Apex Textiles Meter: MT-9981	
Energy Charge : 12000.0 kWh x 8.5 = 102000.0	
Demand Charge : 500.0 kW x 150.0 = 75000.0	
TOTAL BILL : 177000.0 BDT	
=== Heavy Industrial Consumer ===	
ID: C002 Name: Steel Works Ltd Address: Chittagong	
Units: 45000.0 kWh Tariff: 9.0 Peak Demand: 1200.0 kW	
Power Factor Penalty: 25000.0 Generator: 500.0 kVA	
Heavy Bill : 689500.0	
===== HEAVY INDUSTRIAL ENERGY BILL =====	
Consumer : Steel Works Ltd	
Energy Charge : 405000.0	
Demand Charge : 240000.0	
PF Penalty : 25000.0	
Generator Load : 500.0 kVA	
Generator Charge : 19500.0	
TOTAL BILL : 689500.0 BDT	
Total Consumers: 2	

20.4 Main Function Code

01.	public class PowerUtilitySystem {
02.	public static void main(String[] args) {
03.	
04.	EnergyMeter em = new EnergyMeter(
05.	"MT-9981", "Smart Meter", "2022-08-10"
06.	);
07.	
08.	IndustrialConsumer ic = new IndustrialConsumer(
09.	"C001", "Apex Textiles", "Gazipur",
10.	em.getMeterSerialNo(),
11.	12000.0, 8.5, 75.0, 500.0, 150.0
12.	);

```
13.
14.     HeavyIndustrialConsumer hc = new HeavyIndustrialConsumer(
15.         "C002", "Steel Works Ltd", "Chittagong",
16.         "MT-7742",
17.         45000.0, 9.0, 80.0, 1200.0, 200.0,
18.         25000.0, 500.0
19.     );
20.
21.     em.displayMeterInfo();
22.
23.     ic.displayConsumerInfo();
24.     System.out.println("Industrial Bill      : " + ic.calculateBill());
25.     System.out.println("With 1000 extra kWh: " + ic.calculateBill(1000));
26.     ic.generateEnergyBill();
27.
28.     hc.displayConsumerInfo();
29.     System.out.println("Heavy Bill          : " + hc.calculateBill());
30.     hc.generateEnergyBill();
31.
32.     System.out.println("Total Consumers: " + EnergyConsumer.totalConsumers);
33. }
34. }
```